



GenCode: A generic data augmentation framework for boosting deep learning-based code understanding

Zeming Dong¹ · Qiang Hu² · Xiaofei Xie³ · Maxime Cordy¹ · Mike Papadakis¹ · Yves Le Traon¹ · Jianjun Zhao⁴

Received: 24 December 2024 / Accepted: 21 January 2026 / Published online: 10 February 2026

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

Pre-trained code models lead the era of code intelligence, with multiple models designed with impressive performance. However, one important problem, data augmentation for code data that automatically helps developers prepare training data lacks study in this field. In this paper, we introduce a generic data augmentation framework, GenCode, to enhance the training of code understanding models. Simply speaking, GenCode follows a generation-and-selection paradigm to prepare useful training code data. Specifically, it employs code augmentation techniques to generate new code candidates first and then identifies important ones as the training data by influence scores. To evaluate the effectiveness of GenCode, we conduct experiments on four code understanding tasks (e.g., code clone detection) and three pre-trained code models (e.g., CodeT5) and two recent released code-specific Large Language Models (LLMs) (e.g., Qwen2.5-Coder). Compared to the *state-of-the-art* (SOTA) code augmentation method MixCode, GenCode produces pre-trained code models with 2.92% higher accuracy and 4.90% adversarial robustness on average. For code-specific LLMs, GenCode achieves an average improvement of 0.93% in accuracy and 0.98% in natural robustness.

Keywords Code understanding · Data augmentation · Program transformation

Communicated by: Zhi Jin.

✉ Qiang Hu
qianghu0515@gmail.com

Zeming Dong
dongzm9312@gmail.com

¹ University of Luxembourg, Esch-sur-Alzette, Luxembourg

² Tianjin University, Tianjin, China

³ Singapore Management University, Singapore, Singapore

⁴ Kyushu University, Kyushu, Japan

1 Introduction

Deep learning (DL) for code has been one of the successful DL applications over the years. The methodology of applying deep learning to solve *software engineering (SE)* tasks, such as software vulnerability detection (Zhou et al. 2019), is commonly known as *code learning*. From the model level, different DL models specifically designed for code learning have been proposed, such as *CodeBERT* (Feng et al. 2020), *CodeT5* (Wang et al. 2021), and *Qwen2.5-Coder* (Hui et al. 2024). From the commercial perspective, multiple related products such as Copilot¹ and Tabnine² have been widely used to help our daily development. As a result, in different downstream learning-based code understanding tasks, e.g., code clone detection (Lu et al. 2021) and program classification (Puri et al. 2021), DL has already achieved significantly better results than traditional SE techniques.

However, most of the existing code learning research focuses on designing new code representation techniques or code model architectures to further improve the effectiveness of DL for code, one important problem – how to prepare high-quality training data gained limited attention even though recent research demonstrated it highly affects the performance of code models (Sun et al. 2022). The challenge behind this is preparing training data for code models is heavy work. Although today developers can easily collect raw code data from the open-source community such as GitHub³, data labeling is still expensive, especially for code-related tasks. It requires huge human effort and professional domain knowledge.

To tackle this problem, data augmentation is a famous technique that increases the diversity and quality of training data based only on existing labeled data. In the code learning field techniques such as code refactoring methods and text-oriented data augmentation methods can be directly used for code augmentation. Unfortunately, existing studies (Bielik and Vechev 2020; Yu et al. 2022) stated that code refactoring methods have limited advantages in the accuracy improvement of code models. Besides, the recent work (Dong et al. 2025) revealed that code models trained using existing data augmentation methods still suffer from the low robustness issue. Therefore, exploring more effective and reliable code augmentation methods is an emerging task.

To bridge this gap, inspired by the great success of the typical two-stage *Automated Data Augmentation (AutoDA)* framework in the field of *computer vision* (Cubuk et al. 2020; Yang et al. 2023b), we propose GenCode, the first generation-and-selection-based data augmentation framework for code understanding. Specifically, at each training epoch, GenCode first generates *training candidates* by using different code augmentation methods. Here, we consider different types of methods such as semantic-preserving code refactoring methods and syntax-breaking text augmentation methods. After that, GenCode selects valuable training data from the candidates with the guidance of influence scores. The influence score used in GenCode is simply defined as the *loss value* of code data produced by the code model. Finally, the selected samples are used for the model training. We conducted comprehensive experiments to study how GenCode can improve the accuracy and robustness of code models. Based on that, we answer the following three research questions:

¹ <https://github.com/features/copilot>

² <https://www.tabnine.com>

³ <https://github.com>

RQ1: How Effective is GenCode in Producing Accurate Code Models? The results demonstrate that, on average, GenCode can enhance the accuracy of pre-trained code models by up to 3.63% and 2.92% compared to the training without using data augmentation and the SOTA method MixCode, respectively. For code-specific LLMs, GenCode achieves average improvements of 0.93% for StarCoder2 and 0.84% for Qwen2.5-Coder. Notably, our statistical tests show GenCode consistently outperforms baseline methods in terms of accuracy improvement.

RQ2: How Effective is GenCode in Producing Robust Code Models? The results demonstrate that GenCode produces more robust pre-trained code models compared to existing data augmentation methods. Specifically, GenCode achieves an average robustness improvement of 8.42% over models trained without data augmentation and 4.90% over models trained with MixCode. However, for code-specific LLMs, the robustness gains are modest, with improvements of less than 1%.

RQ3: How Does the Influence Score Affect the Effectiveness of GenCode? The results suggest that using the influence score based on maximum loss values is the most effective strategy among the considered variants, including minimum loss value selection and random selection. Besides, even GenCode with random selection outperforms existing code augmentation methods.

RQ4: How Does the Data Selection Method Affect the Effectiveness of GenCode? Experimental results indicate that GenCode achieves superior performance in terms of accuracy improvement. However, the gradient-based method outperforms GenCode and other data selection methods in enhancing model robustness.

To summarize, the main contributions of this paper are:

We propose the first generation-and-selection-based framework, GenCode⁴, which automatically enriches the training data for boosting the code understanding models.

We empirically demonstrate that GenCode can produce more accurate and robust code understanding models compared to existing data augmentation methods.

We conduct an ablation study to explore how different influence scores and data selection methods affect the effectiveness of GenCode.

2 Background

In this section, we introduce the necessary knowledge that is needed to understand our work, code representation, data augmentation for code understanding, and robustness testing of code model.

⁴<https://github.com/zemingd/GenCode>

2.1 Code Representation

Code representation is a crucial technique that converts source code into a format interpretable by *Deep Neural Networks (DNNs)*, facilitating effective learning of source code features. Since source code can be seen as sequences of tokens or structural graph (Allamanis et al. 2018), researchers employ the techniques in *Natural Language Processing (NLP)* and *Graph Neural Networks (GNNs)* to learn information from code and solve downstream tasks, such as automated program classification (Puri et al. 2021) and code clone detection (Wang et al. 2020). Existing methods (Allamanis et al. 2018) for code representation can be roughly divided into two groups, sequence-based representation and graph-based representation. *Sequence-based representation* follows the same procedure of text data processing in NLP to convert raw code data into a sequence of code tokens (Alon et al. 2019a). *Graph-based representation* learns the structural information of code. Here, the structural information can be *abstract syntax tree (AST)*, data flow information, and other types of flow information (Dinella et al. 2020; Yasunaga and Liang 2020).

In this work, we focus on code data preprocessing and study the powerful pre-trained code models and code-specific LLMs that achieved impressive results on code understanding tasks.

2.2 Data Augmentation for Code Understanding

Data augmentation is a well-studied technique in the machine learning field that increases the size and diversity of training data by only modifying the existing labeled data, thus, without further human effort. For instance, in the NLP field, researchers tried many ways to augment the text data, e.g., replacing a token with a synonym (Feng et al. 2021).

Recently, data augmentation has gained attention from software engineering researchers. Various data augmentation methods designed for source code have been proposed (Gao et al. 2020; Yefet et al. 2020; Allamanis et al. 2021; Pour et al. 2021; Bui et al. 2021; Wang et al. 2022; Park et al. 2023; Yang et al. 2023a; Dong et al. 2024a), with the most general framework involving the use of code refactoring, e.g., *local variable renaming* and *dead store*, to create more code data without breaking the semantics of the original codes. Concretely, code refactoring rewrites the syntactic structure of source code while keeping the semantic information (Lacerda et al. 2020).

Existing works (Bielik and Vechev 2020; Yu et al. 2022; de Paula Rodrigues et al. 2023; Dong et al. 2025, 2024b) conducted empirical studies and found code refactoring methods have limited advantages in boosting the training of code models. For instance, a recent empirical study (Dong et al. 2024b) on code understanding revealed that the utilization of code refactoring methods only achieved success rates of 66.67% and 41.67% in improving accuracy and robustness, respectively, compared to code model training without data augmentation. Besides, the SOTA code augmentation method, MixCode, which linearly interpolates pairs of source codes, has been proven more effective than the current general framework code refactoring. However, as a variant of Mixup specifically designed for code data, MixCode is limited to applications in code classification tasks, as it necessitates the label with one-hot encoding.

However, existing studies (Bielik and Vechev 2020; Yu et al. 2022) have stated that these code refactoring methods have limited advantages in boosting the training of code models.

To solve this problem, we propose a simple yet effective code augmentation framework that is not only unaffected by the dataset and model constraints but also applicable to both classification and non-classification tasks.

2.3 Robustness Testing of Code Model

Robustness is an important property of DL models which indicates how the DL model can handle data with different distribution of the training data. Normally, there are two types of robustness of DL models, natural robustness and adversarial robustness. Natural robustness indicates how models perform when facing the natural distribution shift, for example, data in different environments and newly collected data. Adversarial robustness refers to the ability of models to defend the adversarial attacks. Robustness testing of code models has been addressed recently. Researchers proposed distribution shifted code (e.g., codes have the same functionality but are written by different programmers) data to study the natural robustness of code models. Besides, multiple adversarial attack methods have been proposed to test the adversarial robustness of code models, for example, the recent work (Yang et al. 2022) proposed to change the variable name of code to force the model to make wrong predictions.

In this work, we employ adversarial attack methods (Dong et al. 2025) proposed for code to evaluate the adversarial robustness of our trained models. Following the existing work (Dong et al. 2023), we construct the natural robustness datasets to evaluate the performance of all code models.

3 Definition

We first define the problem of data augmentation for code models.

Definition 1 (Program Code) A program code⁵ is defined as a 5-tuple $\mathcal{P} = (content, se, sy, Ip, Op)$: *content* contains all the text in the code $\mathcal{P}$; *se* and *sy* are the semantic and syntax of the code $\mathcal{P}$; and *Ip* and *Op* are the input and output of the code $\mathcal{P}$.

Definition 2 (Code Model) A code model is defined as a 3-tuple $M = (parameter, \mathcal{P}, Y)$, where *parameter* contains all the weights in code model M , $\mathcal{P}$ is the input code, and Y is the label of $\mathcal{P}$ given a specific downstream code task.

Code models can be any machine learning model such as a simple regression model or a pre-trained model, e.g., CodeT5 (Wang et al. 2021).

Definition 3 (Code Augmentation) Given a code $\mathcal{P}$, a code augmentation method $\mathcal{T}: \mathcal{X} \rightarrow \mathcal{X}'$, code augmentation is to transform $\mathcal{P}$ to $\mathcal{P}'$ that $\mathcal{P}'.content = \mathcal{T}(\mathcal{P}.content) \ \& \ \mathcal{P}'.content \neq \mathcal{P}.content$.

⁵For simplicity, we use *code* in the paper.

Here, code augmentation methods can be any method that can change the content of program statements such as text-oriented data augmentation methods (e.g., synonym replacement and code refactoring methods (e.g., dead if statement adding)).

Definition 4 (Semantic-Preserving (Breaking)) Given a code model M , a code $\mathcal{P}$ and its augmented version $\mathcal{P}'$, semantic-preserving (breaking) code augmentation indicates $\mathcal{P}.se = \mathcal{P}'.se \ \& \ M(\mathcal{P}) = M(\mathcal{P}')$ ($\mathcal{P}.se \neq \mathcal{P}'.se \ \& \ M(\mathcal{P}) \neq M(\mathcal{P}')$).

Definition 5 (Syntax-Preserving (Breaking)) Given a code $\mathcal{P}$ and its augmented version $\mathcal{P}'$, syntax-preserving (breaking) code augmentation indicates $\mathcal{P}.sy = \mathcal{P}'.sy$ ($\mathcal{P}.sy \neq \mathcal{P}'.sy$).

4 Assumption and Preliminary Study

GenCode utilizes the influence score to select the most valuable data for the code model training, where the influence score is defined as the loss value. Therefore, this approach is grounded on the assumption that the data that code models have less confidence in is more useful for model training. To check whether this assumption is valid or not, we conducted a preliminary study to explore if there is a correlation between the accuracy of the trained code model and the confidence of the model on the training data. Specifically, the study is conducted on the *Google Code Jam (GCJ)* (Yang et al. 2022) dataset and CodeBERT pre-trained code model. The detailed steps are as follows:

Step 1. We initialize the code model and use code augmentation methods to generate additional data as the training candidate set. The code augmentation methods will be introduced in a later section.

Step 2. We randomly select a number of data from the training candidate set. Here the selected number is the same as the size of the original training data. We repeat this selection 100 times and collect 100 groups of training data. After that, we compute and record the average loss values for each group of training data.

Step 3. We train the code model using each group of training data respectively for a few epochs and thus save 100 trained code models. We then test each code model using the original test data and save 100 accuracies.

Step 4. We use the *Pearson Correlation Coefficient (PCC)* (Schober et al. 2018) metric to check the correlation between the loss values and the accuracies.

We consider three training stages, which are the Early stage (Initialized, Epochs 0–9), the Mid stage (Middle, Epochs 10–19), and the Late stage (Well-trained, Epochs 20–29). Fig. 1 visualizes the results of our preliminary study. Here, we considered three types of code models, the initialized model, the model that has been trained for a few epochs but not converged, and the well-trained code model, to check if our assumption can stand in different scenarios.

The results indicate that there is a positive correlation between the loss values of data and the accuracy of code models trained by using these data. From the initialized model to the well-trained model, this correlation becomes weaker but still exists. We conjecture that when the model is in the early stage, most of the data that have high loss values

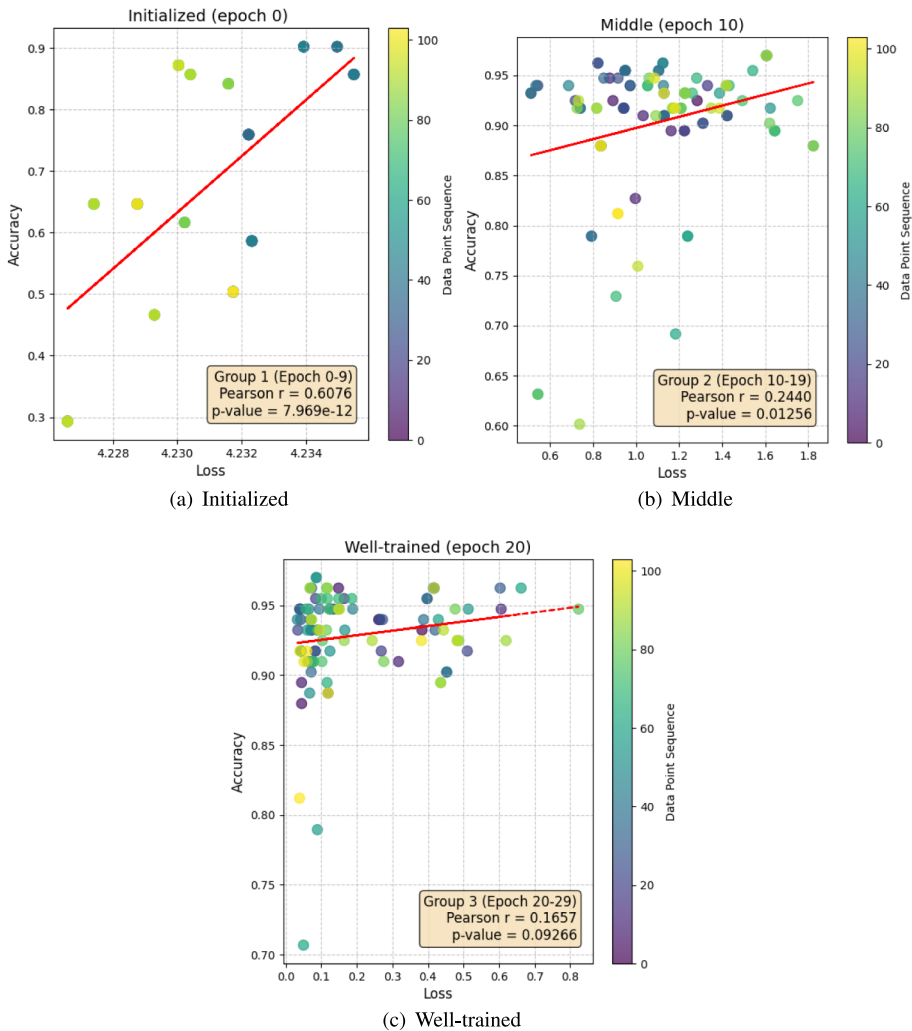


Fig. 1 Correlation between loss values and code model accuracy

with more informative and lead to better model generalization. However, when training proceeds, the data that still have high loss values are the model itself cannot learn. As training progresses, the correlation between loss values and accuracy gradually weakens, which may be attributed to overfitting or the model's limited capacity to learn from inherently difficult samples. This observation supports the hypothesis that the strength of this correlation is indicative of the model's generalization ability. In particular, the decline in correlation from early to later training stages highlights the shifting informativeness of high-loss samples. Leveraging this insight, we design a data augmentation framework that prioritizes informative samples, especially in the early stages, to enhance the training effectiveness of code models.

Finding: There is a positive correlation between the loss values of code data and the accuracy of code models trained by using these data. When the code model is randomly initialized, this correlation is significant (with a *P-value* less than 0.01).

5 Methodology

5.1 Overview

The goal of GenCode is to produce code models with better performance on test data by training models on carefully generated data. Given a training set of program $\mathcal{P}_{train}$, its transformed version $\mathcal{P}'_{train}$, and a set of test programs $\mathcal{P}_{test}$, after the training process $\text{train}(\cdot)$, $\varrho(M'(\mathcal{P}_{test})) > \varrho(M(\mathcal{P}_{test}))$, where $M.\text{parameter} = \text{train}(M, \mathcal{P}_{train})$ & $M'.\text{parameter} = \text{train}(M', \mathcal{P}'_{train})$, and $\varrho(\cdot)$ is the performance measurement.

GenCode follows the generation-and-selection workflow and utilizes different types of code augmentation methods to generate diverse code data first, then selects code data that are most useful for training as the training data. Algorithm 1 shows how GenCode works. In each training epoch, GenCode automatically generates more code data (Line 3 to Line 5, Definition 3) and then selects the most important ones (Line 6 to Line 9) to train the code model (Line 10). Here, the data importance is measured by the loss values produced by the code model which is the most common indicator to check how the model performs on the data.

Algorithm 1 GenCode.

Require: M : initialized DNN model
Require: $\mathcal{P}, Y$: original code data and labels
Require: $\mathcal{T} = \{t\}$: a set of code augmentation methods
Ensure: M : trained model

```

1: for  $run \leftarrow \{0, \dots, \#epochs\}$  do
2:    $\mathcal{P}', Y' \leftarrow \phi, \phi$  ▷ initialization
3:   for  $t \in \mathcal{T}$  do
4:      $\mathcal{P}' \leftarrow \mathcal{P}' \cup t(\mathcal{P})$ 
5:      $Y' \leftarrow Y' \cup Y$  ▷ code augmentation
6:    $I \leftarrow \text{Importance}(M, \mathcal{P}')$  ▷ computing importance
7:    $Index \leftarrow \text{argSort}(I)[:, ||\mathcal{P}||]$  ▷ sort by importance
8:    $X_{train} \leftarrow \mathcal{P}'[Index]$ 
9:    $Y_{train} \leftarrow Y'[Index]$ 
10:   $M \leftarrow \text{Train}(M, X_{train}, Y_{train})$ 
11: return  $M$ 

```

More specifically, Fig. 2 depicts the workflow of GenCode in one training epoch. First, a series of code augmentation methods are applied to the original dataset to generate an augmented dataset (*search space*). Then, samples are selected based on their influence score, i.e., the loss during model training (*importance-guided data selection*), ensuring the selected dataset keeps the same size as the original dataset for a fair comparison.

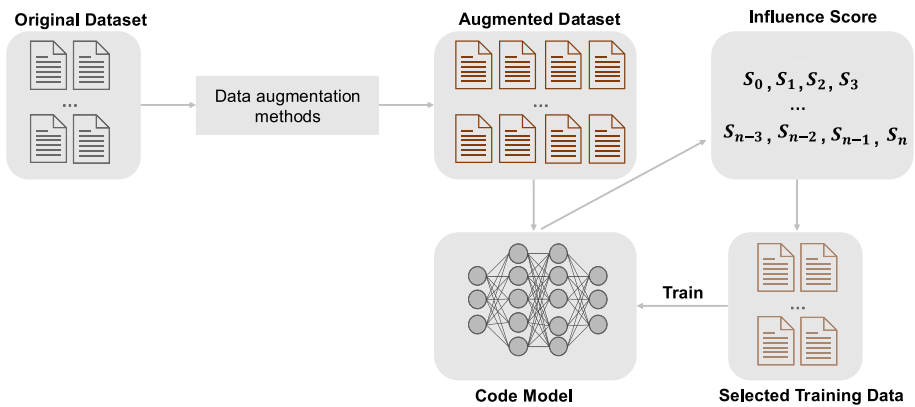


Fig. 2 Workflow of GenCode in one training epoch

Overall, GenCode contains two main steps, utilizing code augmentation methods to compose the search space and importance-guided data selection (will be introduced in Section 5.2 and Section 5.3, respectively).

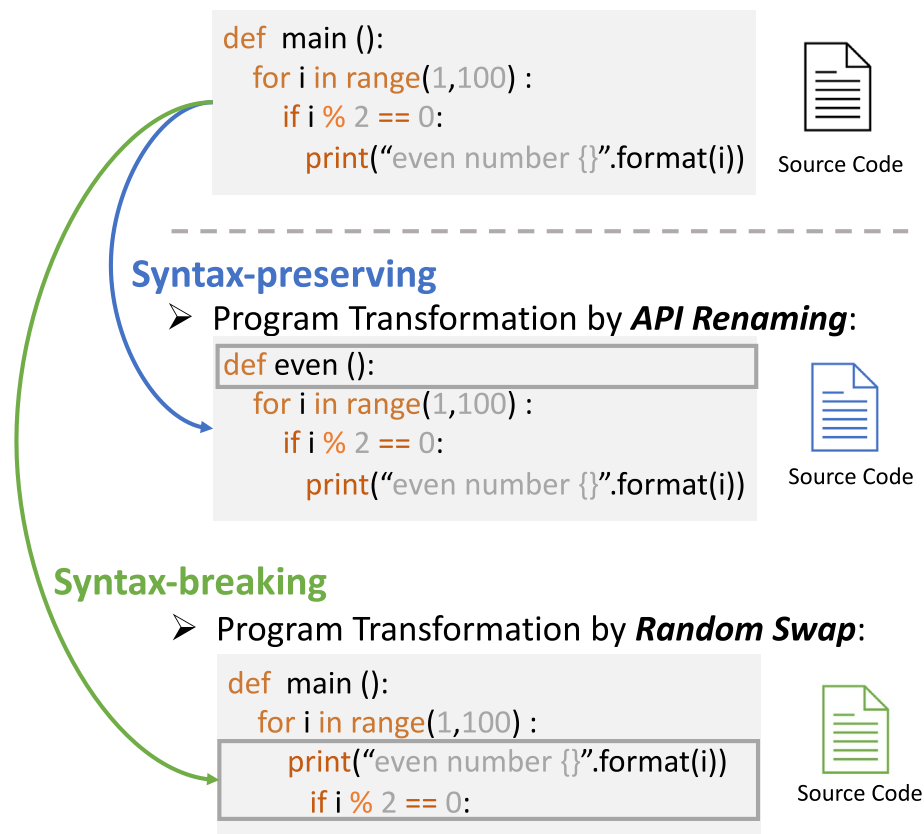
5.2 Search Space

Even though the code has strict syntax constraints, recent work (Dong et al. 2025) found that these data augmentation methods that slightly break the syntax of the source code e.g., *Random Swap*, can still be useful for code model training. To enhance the diversity of source code within the feature space, GenCode considers two types of code augmentation methods, semantic/syntax-preserving methods, and syntax-breaking methods (see Definition 4 and Definition 5). For instance, synonym replacement (SR) may turn an identifier `"|count|"` into a natural-language word, such as `"|amount|"`, which is not declared in the symbol table and thus results in a compilation error. In contrast, refactoring-based renaming updates all references consistently and guarantees syntactic correctness.

GenCode adapts code refactoring methods (i.e., API renaming and Duplication) as semantic/syntax-preserving methods. Code refactoring methods follow the constraints of programming languages (PL) to edit the code while keeping the main functionality of the code. On the other hand, text-oriented data augmentation methods (Dong et al. 2025) are used as syntax-breaking methods in GenCode, which are widely used in the NLP field. For example, text-oriented data augmentation methods can randomly delete some tokens in the code. However, even though the syntax has been broken, the code model can still successfully predict those codes since it reads the code as a sequence. In total, as shown in Table 1, GenCode currently supports 18 code refactoring methods and 5 text-oriented data augmentation methods. Besides, Fig. 3 provides an example of the training candidate that is to generate the search space. In the syntax-preserving, we present a code refactoring method named *API Renaming*, which involves modifying the function name from `"|main()|"` to `"|even()|"`. In the syntax-breaking, we demonstrate another instance of *Random Swap*, where two different statements within a program are randomly selected and their positions swapped. As depicted in this example, the third and fourth lines have been interchanged.

Table 1 Atomic data augmentation operators adopted in GenCode

Syntax-preserving	Syntax-breaking	
API renaming	Field enhancement	
Arguments adding	For loop enhancement	
Arguments renaming	If enhancement	Synonym replacement (SR)
Dead for adding	Local variable adding	Random insertion (RI)
Dead if adding	Local variable renaming	Random swap (RS)
Dead if-else adding	Method name renaming	Random deletion (RD)
Dead switch adding	Plus zero	Back-translation (BT)
Dead while adding	Print adding	
Duplication	Return optimal	

**Fig. 3** An example of atomic data augmentation operators

5.3 Importance-Guided Data Selection

After constructing the training candidate, we then choose the most valuable code data that can benefit the code model training. Based on our *Finding* drawn from the preliminary study, GenCode chooses the simplest way to measure the importance of code data to the model – the *loss* value. That means, GenCode takes all the transformed code as input to the code model and records all the loss values first, then ranks all the code in descending order based on the loss values and selects the first K code as the training data. Following the common setting in the data augmentation problem (Dong et al. 2023), K is set as the number of original training data. GenCode chooses loss value as the indicator since it can be generalized to any code model and task.

6 Evaluation Setup

We evaluate both the accuracy and robustness of trained code models. Besides, we check how does influence score affects the effectiveness of GenCode.

6.1 Study Design

We conduct experiments to evaluate the usefulness of GenCode and answer the following research questions.

RQ1 *How effective is GenCode in producing accurate code models?* Accuracy is the fundamental metric that is used to evaluate the performance of a trained model. Therefore, we first study the basic DNN property, the test accuracy, of each selected pre-trained code model. Specifically, we compare GenCode with models trained without using data augmentation and models trained by baseline data augmentation methods (will be introduced in Section 6.4), and we additionally provide the evaluation results of text-oriented data augmentation methods (as mentioned in Section 5.2) to demonstrate the usefulness of GenCode. Additionally, to ensure a fair comparison in terms of training cost, we apply each data augmentation method to every sample in the original training dataset, resulting in an augmented dataset of the same size. These augmented datasets are fixed prior to training and remain unchanged across all training epochs.

RQ2 *How effective is GenCode in producing robust code models?* Robustness is another important property of DNN models. In this study, we mainly focus on adversarial robustness and natural robustness. Specifically, for the adversarial robustness, we utilize two widely used adversarial attack methods including MHM (Zhang et al. 2020) and ALERT (Yang et al. 2022), to assess the performance of code models trained by the data augmentation method in robustness improvement. For natural robustness, we randomly apply a single code refactoring method (Dong et al. 2023) to each sample in order to construct a transformed code dataset that preserves the original semantic labels, thereby enabling a comprehensive evaluation of the generalization capability under natural variations.

RQ3 *How does the influence score affect the effectiveness of GenCode?* The importance of score-based data selection is the key component is GenCode. Therefore, in this RQ, we change the selection strategies, i.e., selecting data with minimum confidence and randomly selecting data, to study the contribution of our proposed selection strategy. To achieve this, we repeat the experiments outlined in RQ1 and RQ2. Subsequently, we investigate how the selection of different scores influences the performance of pre-trained models. Note that, for a fair comparison, we evaluate only the adversarial robustness, as the use of *random* may artificially inflate performance on the transformed dataset (Dong et al. 2023).

RQ4 *How does the data selection method affect the effectiveness of GenCode?* Data selection strategies play a critical role in determining the performance of trained code models (Hu et al. 2024). In this RQ, we select representative methods from the literature to serve as baselines for comparison with GenCode. Specifically, we include *LIME* (Ribeiro et al. 2016), which provides interpretable, local approximations of complex model predictions and can be leveraged during the model training phase to inform data selection by identifying influential samples. As one of the classical data selection methods, *Influence Function* (Koh and Liang 2017) estimates the effect of individual training samples by approximating how infinitesimal perturbations to the training data impact the learned parameters, and can be effectively utilized during the phase of model training to guide data selection. Additionally, we incorporate *BADGE* (Ash et al. 2019), which selects data by applying K-means clustering on gradient embeddings, and has been shown to be a stable and effective active learning method in recent empirical studies (Hu et al. 2024). We evaluate the performance of selected data selection methods in terms of both accuracy and natural robustness. Adversarial robustness is not considered in this study, as existing adversarial attacks in RQ2 are not well-suited for LLMs.

Table 2 Details of tasks, datasets, and DNN models

Dataset	Language	Task	#Training	#Test	Model
Google Code Jam (GCJ)	Python	Author-ship attribution	528	132	CodeBERT
					GraphCodeBERT
					CodeT5
Refac-tory	Python	Bug detection	3,380	423	StarCoder2-7B
					Qwen2.5-Coder-7B
Java250	Java	Problem classification	48,000	15,000	CodeBERT
					GraphCodeBERT
					CodeT5
					StarCoder2-7B
					Qwen2.5-Coder-7B
#Training and #Test are the number of data for training and testing, respectively	Big-Clone-Bench	Clone detection	90,102	4,000	

6.2 Datasets and Models

We evaluate GenCode on four code understanding tasks as shown in Table 2, bug detection (Hu et al. 2019), authorship attribution (Yang et al. 2022), code clone detection (Yang et al. 2022) (Note that, MixCode is not suitable for this task since its input is code pairs.), and problem classification (Puri et al. 2021). We conduct all experiments on a server equipped with 4 NVIDIA RTX A6000 GPUs. For each code task, we fine-tune three types of models based on pre-trained code models, CodeBERT, GraphCodeBERT, and CodeT5. For the model training, we follow existing works (Hu et al. 2019; Wang et al. 2021; Puri et al. 2021; Yang et al. 2022; Dong et al. 2023), to set all configurations. Specifically, we set the training epoch as 50 for all tasks and models. Regarding the optimizer, we utilize *Adam* (Kingma and Ba 2014) with a learning rate of 10^{-3} for all the models. Besides, we use early stopping with a patience of 20 to mitigate overfitting. To reduce the influence of randomness, we train each model five times and report the average results along with the standard deviation. For MixCode, we follow the recommendation of the original paper and set the Mixup ratio as $\alpha = 0.1$. For the setting of adversarial attacks, we directly use the default configurations provided by the original papers (Yang et al. 2022). Considering the code-specific Large Language Models (LLMs), we select *StarCoder2* (Lozhkov et al. 2024), a representative model from the *BigCode*⁶ project, which is a collaborative initiative dedicated to the responsible development of LLMs for Code, alongside the recently released *Qwen2.5-Coder* (Hui et al. 2024).

6.3 Configuration

We conduct all experiments on a computation server equipped with $4 \times$ NVIDIA A100 (40GB) GPUs, two Intel Xeon Gold 6330 CPUs, and 512GB RAM. To ensure fairness and consistency across benchmarks, we adopt a unified training configuration following established practices in prior work (Wang et al. 2021; Dong et al. 2023; Hu et al. 2019; Yang et al. 2022; Puri et al. 2021). Specifically, we train CodeBERT, GraphCodeBERT, and CodeT5 for 50 epochs, and train StarCoder2-7B and Qwen2.5-Coder-7B for 20 epochs, using the Adam optimizer (learning rate = $1e-3$, weight decay = 0.01). The batch size is fixed to 32, and the maximum sequence length is set to 512 tokens for all encoder-based architectures. To mitigate overfitting and ensure stable convergence, we apply early stopping with a patience of 20 epochs. Each experiment is executed five independent times, and we report the mean and standard deviation to account for stochastic variability. For MixCode, we follow the settings recommended in the original paper and adopt a mixup coefficient of $\alpha = 0.1$. For adversarial training baselines, we directly use the default hyperparameters released by their authors to avoid unintended deviations from the original configurations.

To support reproducibility, we provide the complete training pipeline, including data preprocessing scripts, model configuration files (e.g., batch size, sequence length, and optimizer parameters), and execution instructions. This ensures that all experimental results presented in this work can be reproduced under the same computational environment and hyperparameter settings.

⁶<https://www.bigcode-project.org>

6.4 Evaluation Metrics and Baselines

In the evaluation, we assess the performance of trained models from two perspectives, namely, *clean accuracy* and *model robustness*. For the robustness evaluation, we check the *attack success rate (ASR)* metric of two well-established adversarial attack methods, namely, *ALERT* (Yang et al. 2022), which uses variable renaming while considering natural semantics, and *MHM* (Zhang et al. 2020), which also randomly replaces variable names but does not account for natural semantics. Metric *ASR* evaluates the performance of models, including CodeBERT, GraphCodeBERT, and CodeT5 (Dong et al. 2025). In addition, we evaluate the natural robustness of all trained models by applying 18 different code refactoring methods to generate diverse program variants. Specifically, following the existing work (Dong et al. 2025, 2023), we construct transformed versions of each code sample in the dataset by randomly applying one of the refactoring methods.

We consider *model training without data augmentation (No Aug)*, *Refactor* (Yu et al. 2022), a family of text-oriented data augmentation methods (Dong et al. 2025), and MixCode as baseline methods. *Refactor*, which keeps code syntax rules, is a widely used data augmentation framework specifically designed for code data. In contrast, text-oriented data augmentation methods, which slightly break syntax rules, have recently proven more effective than Refactor. Additionally, MixCode is a more recent and high-performing code augmentation method that significantly enhances code models.

7 Results Analysis

7.1 RQ1: How Effective is GenCode in Producing Accurate Code Models?

Overall Accuracy Column *Accuracy* in Table 3 summarizes the final test accuracy of code models trained by using different code augmentation methods including GenCode. Surprisingly, from the results, we can see that most of the existing code augmentation methods except *RS* and MixCode negatively affect the performance of code models. This finding is similar to the conclusion from recent code augmentation studies (Bielik and Vechev 2020; Yu et al. 2022; Dong et al. 2025) which states code augmentation has limited advantage in boosting the training of code models and also indicates that data augmentation for code is a challenging problem.

Then, compared to the existing data augmentation methods, the results demonstrate that GenCode consistently enhances the training and achieves the best results across all the considered datasets and models. Concretely, on average, for pre-trained code models, GenCode generates models with 4.52%, 3.05%, 2.56%, and 2.11% higher accuracy than *No Aug* in GCJ, Refactory, Java250, and BigCloneBench, respectively. Besides, compared to MixCode, GenCode can achieve 4.1%, 2.34%, and 2.33% better results in GCJ, Refactory, and Java250 datasets, respectively. Moving to LLMs, GenCode improves the performance of StarCoder2 across all tasks by up to 0.93% on average, and achieves a 0.84% average improvement for Qwen2.5-Coder.

Convergence Speed Analysis Figure 4 depicts the average test accuracy at each training epoch of CodeBERT using different data augmentation methods. The results demonstrated

Table 3 Effectiveness of data augmentation methods w.r.t. test accuracy $\uparrow$ (average $\pm$ standard deviation, %) on original test data

Model	DA method	GCI	Refactory	Java250	BigCloneBench
CodeBERT	No Aug	90.96 $\pm$ 0.07	96.21 $\pm$ 0.13	96.36 $\pm$ 0.09	96.91 $\pm$ 0.12
	Refactor	91.74 $\pm$ 0.11	95.95 $\pm$ 0.19	96.44 $\pm$ 0.12	96.97 $\pm$ 0.21
	SR	70.66 $\pm$ 0.11	96.71 $\pm$ 0.11	96.21 $\pm$ 0.02	97.02 $\pm$ 0.15
	RI	59.91 $\pm$ 0.21	96.43 $\pm$ 0.18	96.15 $\pm$ 0.06	97.38 $\pm$ 0.23
	RS	93.24 $\pm$ 0.13	96.73 $\pm$ 0.15	96.49 $\pm$ 0.04	96.92 $\pm$ 0.12
	RD	76.98 $\pm$ 0.09	96.46 $\pm$ 0.18	96.61 $\pm$ 0.12	97.11 $\pm$ 0.19
	BT	82.74 $\pm$ 0.15	94.31 $\pm$ 0.38	96.06 $\pm$ 0.06	97.13 $\pm$ 0.17
	MixCode	92.21 $\pm$ 0.25	96.76 $\pm$ 0.31	96.62 $\pm$ 0.23	-
	GenCode	96.23 $\pm$ 0.18	99.32 $\pm$ 0.14	98.89 $\pm$ 0.09	98.93 $\pm$ 0.11
	No Aug	93.96 $\pm$ 0.15	96.84 $\pm$ 0.16	96.41 $\pm$ 0.11	96.88 $\pm$ 0.09
	Refactor	91.72 $\pm$ 0.21	95.49 $\pm$ 0.21	96.53 $\pm$ 0.15	97.11 $\pm$ 0.23
	SR	89.46 $\pm$ 0.13	95.57 $\pm$ 0.13	96.46 $\pm$ 0.14	97.14 $\pm$ 0.16
	RI	80.47 $\pm$ 0.21	94.51 $\pm$ 0.25	96.52 $\pm$ 0.09	97.35 $\pm$ 0.17
	RS	94.76 $\pm$ 0.15	97.88 $\pm$ 0.15	96.44 $\pm$ 0.14	97.54 $\pm$ 0.12
GraphCodeBERT	RD	91.71 $\pm$ 0.26	96.53 $\pm$ 0.19	96.56 $\pm$ 0.21	97.16 $\pm$ 0.21
	BT	80.47 $\pm$ 0.05	95.99 $\pm$ 0.27	96.39 $\pm$ 0.13	97.11 $\pm$ 0.16
	MixCode	94.19 $\pm$ 0.25	97.29 $\pm$ 0.15	96.89 $\pm$ 0.19	-
	GenCode	97.93 $\pm$ 0.14	99.45 $\pm$ 0.17	99.16 $\pm$ 0.13	99.23 $\pm$ 0.08
	No Aug	95.02 $\pm$ 0.07	97.22 $\pm$ 0.17	96.98 $\pm$ 0.08	97.38 $\pm$ 0.13
	Refactor	92.21 $\pm$ 0.14	96.43 $\pm$ 0.22	96.92 $\pm$ 0.11	97.24 $\pm$ 0.12
	SR	90.14 $\pm$ 0.21	94.67 $\pm$ 0.14	96.53 $\pm$ 0.05	97.31 $\pm$ 0.08
	RI	78.77 $\pm$ 0.05	95.02 $\pm$ 0.15	96.11 $\pm$ 0.09	97.46 $\pm$ 0.13
	RS	95.66 $\pm$ 0.14	97.97 $\pm$ 0.29	96.34 $\pm$ 0.11	97.03 $\pm$ 0.15
	RD	92.01 $\pm$ 0.11	97.52 $\pm$ 0.26	97.21 $\pm$ 0.07	97.43 $\pm$ 0.07
	BT	81.45 $\pm$ 0.13	95.38 $\pm$ 0.11	96.45 $\pm$ 0.14	97.65 $\pm$ 0.14
	MixCode	94.32 $\pm$ 0.16	97.32 $\pm$ 0.13	96.91 $\pm$ 0.11	-
	GenCode	98.85 $\pm$ 0.11	99.64 $\pm$ 0.06	99.38 $\pm$ 0.11	99.34 $\pm$ 0.12
	No Aug	98.91 $\pm$ 0.02	98.12 $\pm$ 0.01	97.83 $\pm$ 0.02	98.75 $\pm$ 0.03
CodeT5	Refactor	96.92 $\pm$ 0.01	97.99 $\pm$ 0.03	97.25 $\pm$ 0.02	98.61 $\pm$ 0.01
	SR	92.36 $\pm$ 0.04	97.27 $\pm$ 0.04	97.27 $\pm$ 0.03	98.46 $\pm$ 0.04
	RI	90.47 $\pm$ 0.05	96.82 $\pm$ 0.03	97.26 $\pm$ 0.02	98.31 $\pm$ 0.02
	RS	97.66 $\pm$ 0.06	97.41 $\pm$ 0.06	98.19 $\pm$ 0.01	98.15 $\pm$ 0.01
	RD	98.33 $\pm$ 0.03	97.02 $\pm$ 0.02	97.72 $\pm$ 0.02	98.36 $\pm$ 0.03
	BT	89.92 $\pm$ 0.02	97.09 $\pm$ 0.05	97.41 $\pm$ 0.02	98.12 $\pm$ 0.05
	MixCode	98.56 $\pm$ 0.04	98.21 $\pm$ 0.08	98.23 $\pm$ 0.03	-
	GenCode	98.93 $\pm$ 0.04	99.65 $\pm$ 0.02	99.41 $\pm$ 0.05	99.36 $\pm$ 0.09
	No Aug	98.95 $\pm$ 0.03	98.19 $\pm$ 0.03	98.11 $\pm$ 0.02	98.78 $\pm$ 0.01
	Refactor	97.22 $\pm$ 0.03	97.51 $\pm$ 0.02	97.31 $\pm$ 0.01	98.32 $\pm$ 0.04
	SR	92.45 $\pm$ 0.02	97.47 $\pm$ 0.03	97.37 $\pm$ 0.04	98.51 $\pm$ 0.05
	RI	90.72 $\pm$ 0.03	96.31 $\pm$ 0.02	98.07 $\pm$ 0.05	98.34 $\pm$ 0.04
	RS	96.89 $\pm$ 0.05	97.52 $\pm$ 0.05	97.31 $\pm$ 0.02	98.02 $\pm$ 0.06
	RD	96.83 $\pm$ 0.01	96.89 $\pm$ 0.04	97.52 $\pm$ 0.03	98.25 $\pm$ 0.04
StarCoder2-7B	BT	91.07 $\pm$ 0.06	97.15 $\pm$ 0.03	98.02 $\pm$ 0.02	98.06 $\pm$ 0.08
	MixCode	98.61 $\pm$ 0.03	98.46 $\pm$ 0.06	98.53 $\pm$ 0.04	-
	GenCode	98.96 $\pm$ 0.04	99.65 $\pm$ 0.01	99.42 $\pm$ 0.04	99.36 $\pm$ 0.05
	No Aug	98.95 $\pm$ 0.03	98.19 $\pm$ 0.03	98.11 $\pm$ 0.02	98.78 $\pm$ 0.01
	Refactor	97.22 $\pm$ 0.03	97.51 $\pm$ 0.02	97.31 $\pm$ 0.01	98.32 $\pm$ 0.04
	SR	92.45 $\pm$ 0.02	97.47 $\pm$ 0.03	97.37 $\pm$ 0.04	98.51 $\pm$ 0.05
	RI	90.72 $\pm$ 0.03	96.31 $\pm$ 0.02	98.07 $\pm$ 0.05	98.34 $\pm$ 0.04
	RS	96.89 $\pm$ 0.05	97.52 $\pm$ 0.05	97.31 $\pm$ 0.02	98.02 $\pm$ 0.06
	RD	96.83 $\pm$ 0.01	96.89 $\pm$ 0.04	97.52 $\pm$ 0.03	98.25 $\pm$ 0.04
	BT	91.07 $\pm$ 0.06	97.15 $\pm$ 0.03	98.02 $\pm$ 0.02	98.06 $\pm$ 0.08
	MixCode	98.61 $\pm$ 0.03	98.46 $\pm$ 0.06	98.53 $\pm$ 0.04	-
	GenCode	98.96 $\pm$ 0.04	99.65 $\pm$ 0.01	99.42 $\pm$ 0.04	99.36 $\pm$ 0.05
	No Aug	98.95 $\pm$ 0.03	98.19 $\pm$ 0.03	98.11 $\pm$ 0.02	98.78 $\pm$ 0.01
	Refactor	97.22 $\pm$ 0.03	97.51 $\pm$ 0.02	97.31 $\pm$ 0.01	98.32 $\pm$ 0.04
	SR	92.45 $\pm$ 0.02	97.47 $\pm$ 0.03	97.37 $\pm$ 0.04	98.51 $\pm$ 0.05
Qwen2.5-Coder-7B	RI	90.72 $\pm$ 0.03	96.31 $\pm$ 0.02	98.07 $\pm$ 0.05	98.34 $\pm$ 0.04
	RS	96.89 $\pm$ 0.05	97.52 $\pm$ 0.05	97.31 $\pm$ 0.02	98.02 $\pm$ 0.06
	RD	96.83 $\pm$ 0.01	96.89 $\pm$ 0.04	97.52 $\pm$ 0.03	98.25 $\pm$ 0.04
	BT	91.07 $\pm$ 0.06	97.15 $\pm$ 0.03	98.02 $\pm$ 0.02	98.06 $\pm$ 0.08
	MixCode	98.61 $\pm$ 0.03	98.46 $\pm$ 0.06	98.53 $\pm$ 0.04	-
	GenCode	98.96 $\pm$ 0.04	99.65 $\pm$ 0.01	99.42 $\pm$ 0.04	99.36 $\pm$ 0.05
	No Aug	98.95 $\pm$ 0.03	98.19 $\pm$ 0.03	98.11 $\pm$ 0.02	98.78 $\pm$ 0.01
	Refactor	97.22 $\pm$ 0.03	97.51 $\pm$ 0.02	97.31 $\pm$ 0.01	98.32 $\pm$ 0.04
	SR	92.45 $\pm$ 0.02	97.47 $\pm$ 0.03	97.37 $\pm$ 0.04	98.51 $\pm$ 0.05
	RI	90.72 $\pm$ 0.03	96.31 $\pm$ 0.02	98.07 $\pm$ 0.05	98.34 $\pm$ 0.04
	RS	96.89 $\pm$ 0.05	97.52 $\pm$ 0.05	97.31 $\pm$ 0.02	98.02 $\pm$ 0.06
	RD	96.83 $\pm$ 0.01	96.89 $\pm$ 0.04	97.52 $\pm$ 0.03	98.25 $\pm$ 0.04
	BT	91.07 $\pm$ 0.06	97.15 $\pm$ 0.03	98.02 $\pm$ 0.02	98.06 $\pm$ 0.08
	MixCode	98.61 $\pm$ 0.03	98.46 $\pm$ 0.06	98.53 $\pm$ 0.04	-
	GenCode	98.96 $\pm$ 0.04	99.65 $\pm$ 0.01	99.42 $\pm$ 0.04	99.36 $\pm$ 0.05

No Aug: without data augmentation. The best results are highlighted in blue. Code understanding tasks include **Authorship attribution** (GCI), **Bug detection** (Refactory), **Problem classification** (Java250), and **Clone detection** (BigCloneBench)

that GenCode consistently outperforms other baseline methods during the whole training process. We can conclude that the choice of the number of training epochs will not affect the advantages of our method over other methods.

Visualizing Principal Component Analysis Furthermore, we explore why models trained by GenCode have higher accuracy than models trained using other methods by analyzing the code representation capability of trained code embeddings. Specifically, 1) first, we extract the code embeddings produced by each trained model using all the test data as code features. 2) Then, since the extracted features are high dimensions and hard to analyze, we reduce the dimensions of code data features using the *Principal Component Analysis (PCA)* (Maćkiewicz and Ratajczak 1993) algorithm. 3) After that, we visualize the

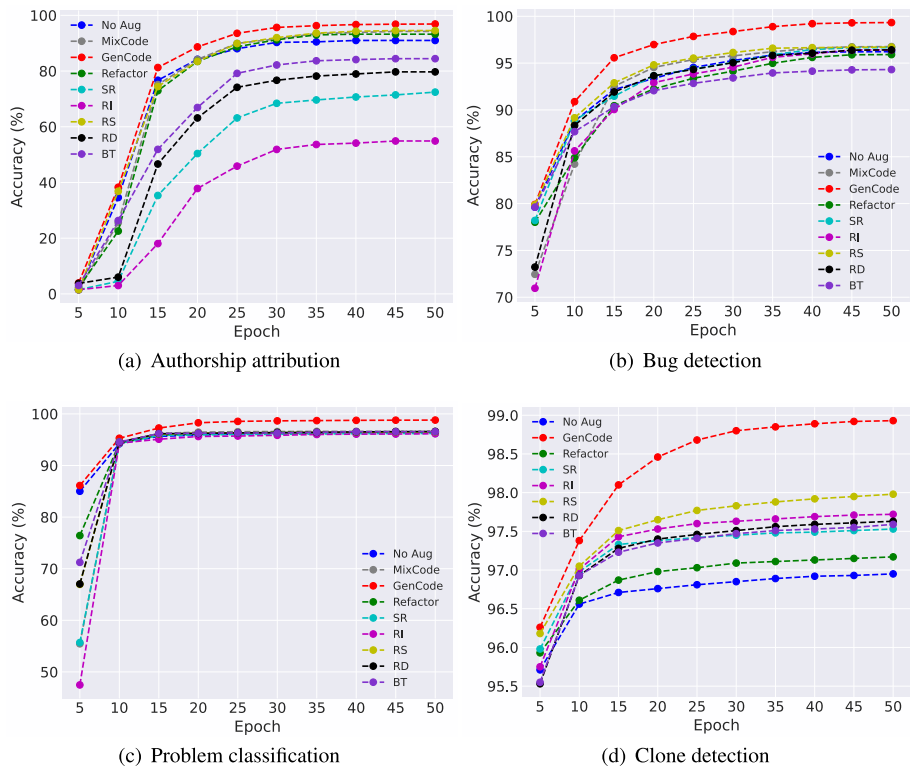


Fig. 4 Convergence speed of CodeBERT using different code augmentation methods in each task

reduced 2-dimensional vectors. Normally, the features extracted by a well-trained model are more clearly distributed based on the data labels.

Figure 5 depicts the visualized features extracted from CodeBERT using the Refactory dataset. We can see that the code embeddings trained by using GenCode are more clearly distributed than embedding extracted by other models, i.e., the buggy code and correct code are greatly different in the figure. These results indicate that GenCode can train good code embeddings and, therefore, produce better accurate code models for code understanding tasks. Besides, we computed the average distance between the data points from different classes, i.e., the average distance between red points and blue points in Fig. 5. The results from Table 4 confirm that GenCode can better distinguish the two classes. Interestingly, we found that MixCode has the smallest distance result. That means even though MixCode can train code models with good performance, the code models have low confidence in their predictions, thus, the data are near the decision boundaries. This phenomenon is similar to the findings in Verma et al. (2019), where interpolating hidden state representations yields two beneficial properties: 1) class representations are flattened into a minimal number of directions of variation, and 2) all points between these flat representations are assigned low-confidence predictions.

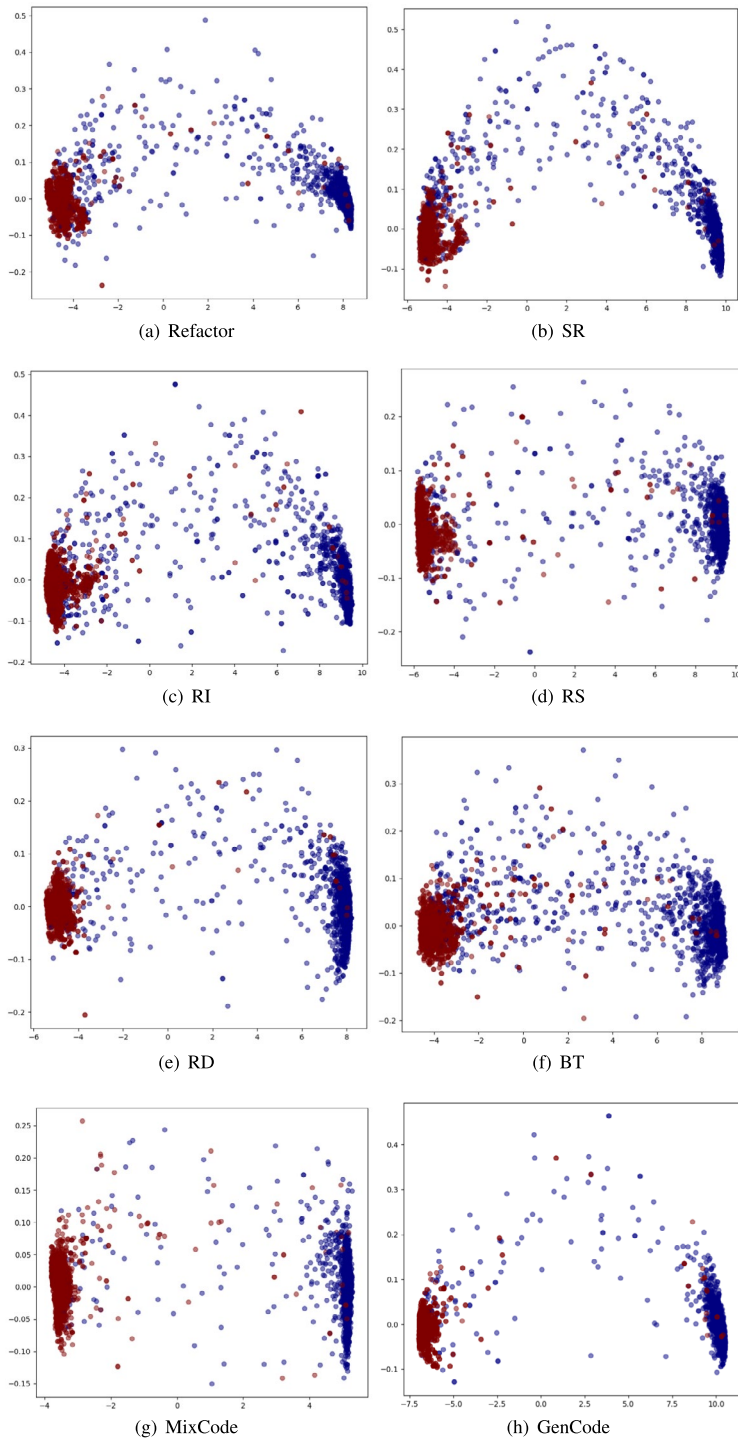


Fig. 5 Visualization of code embeddings after dimension reduction using Principal Component Analysis (PCA). Model: CodeBERT, dataset: Refactory, task: Bug detection

Table 4 Average distance between the data points from different classes

DA method	Refactor	SR	RI	RS
Average distance	6.02	8.05	7.02	8.06
DA method	RD	BT	MixCode	GenCode
Average distance	7.03	6.00	4.05	8.78

The best is highlighted with a blue background, and the worst is highlighted with a gray background

Table 5 Results of statistical tests on Accuracy & Natural Robustness

	DA method	No Aug	Refactor	SR	RI	RS	RD	BT	MixCode
Accuracy	GenCode	1.907e-06	1.907e-06	1.907e-06	8.845e-05	1.907e-06	1.907e-06	1.907e-06	0.001
Natural Robustness	GenCode	1.907e-06	-	1.907e-06	1.907e-06	1.907e-06	1.316e-04	1.907e-06	6.533e-04

A gray background highlights that the comparison is statistically significant. Statistical tests method: *Wilcoxon signed-rank test*. Across CodeBERT, GraphCodeBERT, CodeT5, StarCoder2, and Qwen2.5-Coder models

Statistical Analysis We conducted statistical tests with *Wilcoxon signed-rank test*, across all code models. The results from the upper of Table 5 show that GenCode significantly (with p -value < 0.05) outperforms other methods including MixCode. In conclusion, GenCode is effective in training accurate code models.

Answer to RQ1: GenCode consistently outperforms baselines in training more accurate code understanding models. Specifically, for pre-trained code models, GenCode yields an average accuracy improvement of 3.06% over models trained without data augmentation and 2.92% over those trained with the SOTA MixCode method. For LLMs, GenCode achieves average improvements of 0.93% for StarCoder2 and 0.84% for Qwen2.5-Coder.

7.2 RQ2: How Effective is GenCode in Producing Robust Code Models?

Adversarial Robustness Evaluation Column *MHM* and *ALERT* in Table 6 summarize the attack success rate of adversarial attack methods against code models. First, we can see that similar to the observation in Accuracy Evaluation, not all data augmentation methods can improve the robustness of code models. This finding is consistent with the existing works (Dong et al. 2025). Specifically, in the baseline methods, only *RS* and *MixCode* have a positive impact on the robustness of code models. Considering our method, GenCode consistently improves the robustness of code models and achieves the best results in 21 out of 24 cases. On average, compared to *No Aug*, models trained by GenCode have an 8.42% lower ASR (higher robustness). This result breaks the previous finding (Bielik and Vechev 2020) that states simply adding additional generated code to the training dataset to train code models has limited benefits in improving their robustness. On the other hand, compared to *MixCode*, GenCode can train code models with 4.90% higher robustness on average.

Natural Robustness Evaluation Table 7 presents the result of the natural robustness evaluation. For a fair comparison, we remove *Refactor* from the column *DA method*. The results

Table 6 Effectiveness of data augmentation methods w.r.t. ASR ↓ (%) on test data

Model	DA method	GCI MHM	ALERT	Refactory MHM	ALERT	Java250 MHM	ALERT	BigCloneBench MHM	ALERT
CodeBERT	No Aug	33.31	56.13	36.31	42.31	40.55	53.37	9.74	24.61
	Refactor	35.56	52.84	32.64	40.89	38.35	52.56	8.55	18.29
	SR	35.46	52.71	39.25	53.91	42.01	53.79	12.52	26.11
	RI	40.78	65.76	38.45	45.57	43.15	55.07	8.31	30.08
	RS	28.26	52.44	38.63	45.98	39.36	50.79	13.41	30.13
	RD	35.65	55.46	30.23	41.32	38.31	52.42	9.58	30.15
	BT	40.38	55.95	62.98	69.87	42.21	53.32	11.29	26.32
	MixCode	29.28	52.53	31.04	41.01	36.14	50.67	-	-
	GenCode	22.33	46.62	23.75	31.25	29.89	40.65	7.12	15.83
	No Aug	31.23	56.02	22.97	30.26	23.16	42.66	4.19	12.22
	Refactor	30.31	56.58	24.26	32.57	23.13	42.42	6.05	6.17
	SR	40.02	66.68	28.39	47.15	22.92	42.18	6.13	10.16
	RI	43.51	65.75	25.08	32.87	22.37	42.33	2.23	6.28
	RS	30.18	54.75	21.85	29.67	22.68	41.98	6.03	11.94
	RD	31.73	60.92	20.35	26.98	22.39	41.84	6.18	8.27
	BT	41.65	62.99	35.56	46.23	22.91	42.32	8.18	14.12
GraphCodeBERT	MixCode	30.26	54.49	14.89	26.53	20.03	42.01	-	-
	GenCode	26.43	48.87	15.78	21.92	18.76	33.81	1.86	4.62
	No Aug	30.78	55.67	22.45	29.69	31.76	49.82	10.32	22.97
	Refactor	30.12	55.93	28.76	35.45	30.66	47.12	9.32	22.05
	SR	39.21	51.84	28.24	36.26	28.65	46.01	15.87	25.98
	RI	41.02	63.46	24.78	33.61	27.14	46.76	10.43	29.76
	RS	29.35	51.74	21.32	27.76	28.45	48.45	14.33	30.43
	RD	31.04	56.85	19.93	24.87	26.09	45.24	9.98	28.87
	BT	39.65	54.24	33.62	43.67	29.47	45.38	10.43	25.34
	MixCode	29.82	42.56	12.49	18.32	28.73	48.87	-	-
CodeT5	GenCode	23.57	44.76	12.13	19.56	21.63	39.76	7.39	12.15

The best results are highlighted in blue. Code understanding tasks include **Authorship attribution** (GCI), **Bug detection** (Refactory), **Problem classification** (Java250), and **Clone detection** (BigCloneBench)

indicate that most text-oriented data augmentation methods (Dong et al. 2025), such as *BT* and *RI*, do not contribute significantly to improving natural robustness. Sampling-based code augmentation methods, such as MixCode, yield limited improvements. For instance, in code multi-classification tasks including *authorship attribution* (GCI), MixCode achieves an average improvement of only 0.56% compared to training without data augmentation. Although GenCode outperforms all baselines in Table 7, the improvement remains limited, particularly for LLMs. For example, GenCode achieves an average improvement of only 0.98% for StarCoder2-7B and 0.84% for Qwen2.5-Coder-7B across all tasks. This observation aligns with findings from existing studies (Dong et al. 2025; Ding et al. 2024), suggesting that the effectiveness of data augmentation diminishes as model scale increases.

Maximum Probability Analysis To explore why GenCode produces a more robust code model, we check the *confidence* of trained models using different code augmentation methods on the original test data. Normally, if the DNN model is more confident in the test data, it is more difficult to attack this model using this data (Wu et al. 2018). To do so, for each dataset and code model, we collect the maximum probability of the corrected classified test data produced by the code model first. Then, we calculate the mean values of these maximum probability scores. The results of the mean values of maximum probabilities are shown in Table 8. We can see that code models trained by GenCode are more confident in the test data than models trained using other code augmentation methods. As a result, GenCode trained code models are harder to attack.

Statistical Analysis We also conduct statistical tests using *Wilcoxon signed-rank test*. When comparing existing data augmentation methods, GenCode consistently exhibits a *p-value*

Table 7 Effectiveness of data augmentation methods w.r.t. test accuracy $\uparrow$ (average $\pm$ standard deviation, %) on transformed test data

Model	DA method	GCJ	Refactory	Java250	BigCloneBench
CodeBERT	No Aug	87.57 $\pm$ 0.13	92.12 $\pm$ 0.09	89.68 $\pm$ 0.16	91.61 $\pm$ 0.18
	SR	62.78 $\pm$ 0.22	89.87 $\pm$ 0.15	85.51 $\pm$ 0.07	86.21 $\pm$ 0.12
	RI	50.35 $\pm$ 0.19	90.23 $\pm$ 0.19	85.21 $\pm$ 0.11	92.02 $\pm$ 0.13
	RS	88.23 $\pm$ 0.13	90.56 $\pm$ 0.12	89.97 $\pm$ 0.14	85.65 $\pm$ 0.11
	RD	69.34 $\pm$ 0.18	93.78 $\pm$ 0.11	90.34 $\pm$ 0.13	91.87 $\pm$ 0.12
	BT	75.43 $\pm$ 0.15	82.35 $\pm$ 0.21	85.23 $\pm$ 0.15	87.78 $\pm$ 0.09
	MixCode	89.11 $\pm$ 0.13	95.36 $\pm$ 0.09	93.19 $\pm$ 0.12	-
	GenCode	91.23 $\pm$ 0.14	95.63 $\pm$ 0.14	93.62 $\pm$ 0.09	92.56 $\pm$ 0.15
	No Aug	87.86 $\pm$ 0.12	92.04 $\pm$ 0.03	90.58 $\pm$ 0.08	91.89 $\pm$ 0.09
	SR	66.45 $\pm$ 0.09	89.12 $\pm$ 0.06	91.05 $\pm$ 0.12	89.23 $\pm$ 0.13
	RI	53.87 $\pm$ 0.13	91.29 $\pm$ 0.07	90.88 $\pm$ 0.06	93.57 $\pm$ 0.15
	RS	87.73 $\pm$ 0.15	92.56 $\pm$ 0.02	91.44 $\pm$ 0.11	89.02 $\pm$ 0.08
	RD	73.59 $\pm$ 0.08	92.87 $\pm$ 0.05	92.01 $\pm$ 0.05	92.21 $\pm$ 0.16
	BT	73.24 $\pm$ 0.11	86.05 $\pm$ 0.11	93.14 $\pm$ 0.04	90.25 $\pm$ 0.12
GraphCodeBERT	MixCode	88.13 $\pm$ 0.13	95.24 $\pm$ 0.12	94.05 $\pm$ 0.15	-
	GenCode	89.78 $\pm$ 0.16	95.67 $\pm$ 0.15	94.89 $\pm$ 0.08	93.89 $\pm$ 0.11
	No Aug	89.31 $\pm$ 0.15	92.89 $\pm$ 0.08	91.13 $\pm$ 0.12	91.93 $\pm$ 0.13
	SR	68.94 $\pm$ 0.06	90.21 $\pm$ 0.12	88.23 $\pm$ 0.08	89.43 $\pm$ 0.08
	RI	58.34 $\pm$ 0.18	91.14 $\pm$ 0.14	91.13 $\pm$ 0.13	93.29 $\pm$ 0.17
	RS	89.98 $\pm$ 0.19	92.89 $\pm$ 0.13	90.87 $\pm$ 0.07	88.34 $\pm$ 0.07
	RD	75.57 $\pm$ 0.13	93.98 $\pm$ 0.12	91.97 $\pm$ 0.11	92.35 $\pm$ 0.14
	BT	75.64 $\pm$ 0.11	88.79 $\pm$ 0.08	94.06 $\pm$ 0.16	90.87 $\pm$ 0.05
	MixCode	88.45 $\pm$ 0.14	95.86 $\pm$ 0.14	93.89 $\pm$ 0.19	-
	GenCode	91.38 $\pm$ 0.09	95.87 $\pm$ 0.11	95.06 $\pm$ 0.12	94.03 $\pm$ 0.12
	No Aug	92.53 $\pm$ 0.02	94.12 $\pm$ 0.05	92.71 $\pm$ 0.11	93.21 $\pm$ 0.04
	SR	82.75 $\pm$ 0.03	92.31 $\pm$ 0.07	91.87 $\pm$ 0.03	91.45 $\pm$ 0.05
	RI	75.89 $\pm$ 0.05	93.19 $\pm$ 0.12	92.35 $\pm$ 0.05	93.88 $\pm$ 0.13
	RS	92.13 $\pm$ 0.06	94.11 $\pm$ 0.08	91.67 $\pm$ 0.14	90.11 $\pm$ 0.06
CodeT5	RD	90.15 $\pm$ 0.08	94.67 $\pm$ 0.09	93.03 $\pm$ 0.02	92.89 $\pm$ 0.09
	BT	85.62 $\pm$ 0.04	90.12 $\pm$ 0.13	92.55 $\pm$ 0.09	91.12 $\pm$ 0.03
	MixCode	92.62 $\pm$ 0.01	95.91 $\pm$ 0.01	92.58 $\pm$ 0.06	-
	GenCode	92.91 $\pm$ 0.03	96.32 $\pm$ 0.12	93.03 $\pm$ 0.04	94.21 $\pm$ 0.12
	No Aug	92.61 $\pm$ 0.03	94.35 $\pm$ 0.11	92.82 $\pm$ 0.14	93.45 $\pm$ 0.06
	SR	85.67 $\pm$ 0.04	92.67 $\pm$ 0.13	91.93 $\pm$ 0.08	91.55 $\pm$ 0.08
	RI	78.94 $\pm$ 0.06	93.01 $\pm$ 0.21	92.65 $\pm$ 0.06	93.75 $\pm$ 0.04
	RS	91.59 $\pm$ 0.07	94.31 $\pm$ 0.09	92.04 $\pm$ 0.02	90.32 $\pm$ 0.09
	RD	90.65 $\pm$ 0.11	95.01 $\pm$ 0.04	92.65 $\pm$ 0.07	93.14 $\pm$ 0.12
	BT	83.65 $\pm$ 0.02	91.23 $\pm$ 0.11	92.86 $\pm$ 0.13	91.56 $\pm$ 0.05
	MixCode	92.89 $\pm$ 0.09	94.89 $\pm$ 0.02	92.67 $\pm$ 0.03	-
	GenCode	93.12 $\pm$ 0.02	96.17 $\pm$ 0.15	92.93 $\pm$ 0.09	94.38 $\pm$ 0.14
	No Aug	92.61 $\pm$ 0.03	94.35 $\pm$ 0.11	92.82 $\pm$ 0.14	93.45 $\pm$ 0.06
	SR	85.67 $\pm$ 0.04	92.67 $\pm$ 0.13	91.93 $\pm$ 0.08	91.55 $\pm$ 0.08
	RI	78.94 $\pm$ 0.06	93.01 $\pm$ 0.21	92.65 $\pm$ 0.06	93.75 $\pm$ 0.04
StarCoder2-7B	RS	91.59 $\pm$ 0.07	94.31 $\pm$ 0.09	92.04 $\pm$ 0.02	90.32 $\pm$ 0.09
	RD	90.65 $\pm$ 0.11	95.01 $\pm$ 0.04	92.65 $\pm$ 0.07	93.14 $\pm$ 0.12
	BT	83.65 $\pm$ 0.02	91.23 $\pm$ 0.11	92.86 $\pm$ 0.13	91.56 $\pm$ 0.05
	MixCode	92.89 $\pm$ 0.09	94.89 $\pm$ 0.02	92.67 $\pm$ 0.03	-
	GenCode	93.12 $\pm$ 0.02	96.17 $\pm$ 0.15	92.93 $\pm$ 0.09	94.38 $\pm$ 0.14
	No Aug	92.61 $\pm$ 0.03	94.35 $\pm$ 0.11	92.82 $\pm$ 0.14	93.45 $\pm$ 0.06
	SR	85.67 $\pm$ 0.04	92.67 $\pm$ 0.13	91.93 $\pm$ 0.08	91.55 $\pm$ 0.08
	RI	78.94 $\pm$ 0.06	93.01 $\pm$ 0.21	92.65 $\pm$ 0.06	93.75 $\pm$ 0.04
	RS	91.59 $\pm$ 0.07	94.31 $\pm$ 0.09	92.04 $\pm$ 0.02	90.32 $\pm$ 0.09
	RD	90.65 $\pm$ 0.11	95.01 $\pm$ 0.04	92.65 $\pm$ 0.07	93.14 $\pm$ 0.12
	BT	83.65 $\pm$ 0.02	91.23 $\pm$ 0.11	92.86 $\pm$ 0.13	91.56 $\pm$ 0.05
	MixCode	92.89 $\pm$ 0.09	94.89 $\pm$ 0.02	92.67 $\pm$ 0.03	-
	GenCode	93.12 $\pm$ 0.02	96.17 $\pm$ 0.15	92.93 $\pm$ 0.09	94.38 $\pm$ 0.14
	No Aug	92.61 $\pm$ 0.03	94.35 $\pm$ 0.11	92.82 $\pm$ 0.14	93.45 $\pm$ 0.06
	SR	85.67 $\pm$ 0.04	92.67 $\pm$ 0.13	91.93 $\pm$ 0.08	91.55 $\pm$ 0.08
Qwen2.5-Coder-7B	RI	78.94 $\pm$ 0.06	93.01 $\pm$ 0.21	92.65 $\pm$ 0.06	93.75 $\pm$ 0.04
	RS	91.59 $\pm$ 0.07	94.31 $\pm$ 0.09	92.04 $\pm$ 0.02	90.32 $\pm$ 0.09
	RD	90.65 $\pm$ 0.11	95.01 $\pm$ 0.04	92.65 $\pm$ 0.07	93.14 $\pm$ 0.12
	BT	83.65 $\pm$ 0.02	91.23 $\pm$ 0.11	92.86 $\pm$ 0.13	91.56 $\pm$ 0.05
	MixCode	92.89 $\pm$ 0.09	94.89 $\pm$ 0.02	92.67 $\pm$ 0.03	-
	GenCode	93.12 $\pm$ 0.02	96.17 $\pm$ 0.15	92.93 $\pm$ 0.09	94.38 $\pm$ 0.14
	No Aug	92.61 $\pm$ 0.03	94.35 $\pm$ 0.11	92.82 $\pm$ 0.14	93.45 $\pm$ 0.06
	SR	85.67 $\pm$ 0.04	92.67 $\pm$ 0.13	91.93 $\pm$ 0.08	91.55 $\pm$ 0.08
	RI	78.94 $\pm$ 0.06	93.01 $\pm$ 0.21	92.65 $\pm$ 0.06	93.75 $\pm$ 0.04
	RS	91.59 $\pm$ 0.07	94.31 $\pm$ 0.09	92.04 $\pm$ 0.02	90.32 $\pm$ 0.09
	RD	90.65 $\pm$ 0.11	95.01 $\pm$ 0.04	92.65 $\pm$ 0.07	93.14 $\pm$ 0.12
	BT	83.65 $\pm$ 0.02	91.23 $\pm$ 0.11	92.86 $\pm$ 0.13	91.56 $\pm$ 0.05
	MixCode	92.89 $\pm$ 0.09	94.89 $\pm$ 0.02	92.67 $\pm$ 0.03	-
	GenCode	93.12 $\pm$ 0.02	96.17 $\pm$ 0.15	92.93 $\pm$ 0.09	94.38 $\pm$ 0.14
	No Aug	92.61 $\pm$ 0.03	94.35 $\pm$ 0.11	92.82 $\pm$ 0.14	93.45 $\pm$ 0.06

No Aug: without data augmentation. The best results are highlighted in blue. Code understanding tasks include **Authorship attribution** (GCJ), **Bug detection** (Refactory), **Problem classification** (Java250), and **Clone detection** (BigCloneBench)

< 0.05 across all pre-trained code models shown from the lower of Table 5. In conclusion, GenCode can produce more adversarial robust code understanding models than other methods.

Answer to RQ2: Our statistical tests find GenCode can train more robust pre-trained code models than existing data augmentation methods. On average, code models trained using GenCode have 8.42% and 4.90% higher robustness than models trained without data augmentation and models trained using MixCode, respectively. However, for LLMs, the robustness improvement achieved by GenCode remains limited, with gains of less than 1%.

Table 8 Mean values of maximum probabilities

		Model	DA method	GCJ	Refactory	Java250	BigCloneBench
The best results are highlighted in blue. Code understanding tasks include Authorship attribution (GCJ), Bug detection (Refactory), Problem classification (Java250), and Clone detection (BigCloneBench)		No Aug	0.0381945	0.9368535	0.9554731	0.9651272	
		Refactor	0.0391864	0.9643155	0.9620814	0.9654681	
		SR	0.0380562	0.9527686	0.9341456	0.9583812	
		RI	0.0371215	0.9189242	0.9327171	0.9704512	
		RS	0.0392987	0.9561591	0.9581767	0.9410937	
		RD	0.0392987	0.9723022	0.9634647	0.9631375	
		BT	0.0379581	0.9566747	0.9417861	0.9667154	
		MixCode	0.0390714	0.9687351	0.9685594	-	
	CodeBERT	GenCode	0.0413779	0.9808122	0.9714346	0.9786421	

Table 9 Effectiveness of data augmentation methods w.r.t. test accuracy $\uparrow$ (average $\pm$ standard deviation, %) on original test data

Model	DA method	GCJ	Refactory	Java250	BigCloneBench
CodeBERT	Random	94.74 $\pm$ 0.21	98.82 $\pm$ 0.15	98.24 $\pm$ 0.11	98.27 $\pm$ 0.19
	Min	93.34 $\pm$ 0.32	96.13 $\pm$ 0.21	96.43 $\pm$ 0.17	97.23 $\pm$ 0.14
	Max	96.23 $\pm$ 0.18	99.32 $\pm$ 0.24	98.89 $\pm$ 0.09	98.93 $\pm$ 0.15
GraphCodeBERT	Random	95.26 $\pm$ 0.11	98.93 $\pm$ 0.18	98.68 $\pm$ 0.22	98.67 $\pm$ 0.13
	Min	93.53 $\pm$ 0.17	96.31 $\pm$ 0.24	96.72 $\pm$ 0.12	97.22 $\pm$ 0.21
	Max	97.93 $\pm$ 0.14	99.45 $\pm$ 0.17	99.16 $\pm$ 0.13	99.23 $\pm$ 0.09
CodeT5	Random	96.24 $\pm$ 0.21	98.99 $\pm$ 0.21	98.93 $\pm$ 0.16	98.83 $\pm$ 0.08
	Min	94.11 $\pm$ 0.31	98.29 $\pm$ 0.12	97.03 $\pm$ 0.23	97.01 $\pm$ 0.15
	Max	98.85 $\pm$ 0.16	99.64 $\pm$ 0.37	99.38 $\pm$ 0.11	99.34 $\pm$ 0.12
StarCoder2-7B	Random	96.81 $\pm$ 0.06	99.11 $\pm$ 0.02	99.23 $\pm$ 0.04	99.31 $\pm$ 0.06
	Min	95.42 $\pm$ 0.03	98.68 $\pm$ 0.04	98.47 $\pm$ 0.03	98.13 $\pm$ 0.03
	Max	98.93 $\pm$ 0.04	99.65 $\pm$ 0.02	99.41 $\pm$ 0.05	99.36 $\pm$ 0.09
Qwen2.5-Coder-7B	Random	97.12 $\pm$ 0.05	99.12 $\pm$ 0.03	99.23 $\pm$ 0.03	99.32 $\pm$ 0.04
	Min	95.86 $\pm$ 0.08	98.71 $\pm$ 0.01	98.51 $\pm$ 0.05	98.19 $\pm$ 0.07
	Max	98.96 $\pm$ 0.04	99.65 $\pm$ 0.01	99.42 $\pm$ 0.04	99.36 $\pm$ 0.05

The best results are highlighted in blue. Code understanding tasks include **Authorship attribution** (GCJ), **Bug detection** (Refactory), **Problem classification** (Java250), and **Clone detection** (BigCloneBench)

Table 10 Effectiveness of data augmentation methods w.r.t. ASR $\downarrow$ (%) on test data

Model	DA method	GCJ MHM	ALERT	Refactory MHM	ALERT	Java250 MHM	ALERT	BigCloneBench MHM	ALERT
CodeBERT	Random	25.21	50.32	27.65	36.65	32.52	44.87	8.01	18.12
	Min	35.66	66.67	65.51	69.76	46.76	53.86	20.65	41.32
	Max	22.33	42.62	22.75	29.25	29.89	40.65	7.12	15.83
GraphCodeBERT	Random	29.97	50.63	17.56	24.29	20.65	38.86	2.04	5.97
	Min	40.55	68.23	38.73	46.98	26.87	46.67	11.45	21.23
	Max	26.43	39.87	15.78	20.92	18.76	33.81	1.86	4.62
CodeT5	Random	25.03	47.61	18.46	23.67	25.11	42.34	8.67	17.42
	Min	38.65	59.89	35.67	48.59	32.61	49.27	18.36	25.54
	Max	23.57	38.76	12.13	18.81	21.63	39.76	7.39	12.15

The best results are highlighted in blue. Code understanding tasks include **Authorship attribution** (GCJ), **Bug detection** (Refactory), **Problem classification** (Java250), and **Clone detection** (BigCloneBench)

7.3 RQ3: How Does the Influence Score Affect the Effectiveness of GenCode?

Even though we have preliminarily shown that selecting the code samples with higher loss values is a valid way to train good models, it is still necessary to study to what extent the advantage can be brought from this selection strategy. Therefore, in this part, we conduct an ablation study to compare GenCode with its two variants, GenCode with data selection

using minimum loss values - GenCode (Min) and random selection - GenCode (Random), respectively.

Tables 9 and 10 summarize the accuracy and robustness of trained code models using different selection strategies. We can see GenCode consistently outperforms the two variants under all testing criteria. Selecting code samples with minimum loss values is the worst way for GenCode. Interestingly, we found that GenCode (Random) performs better than all baseline methods (i.e., *No Aug*, Refactor, and MixCode) in terms of both accuracy and robustness improvements based on the results.

This phenomenon indicates that simply combining all syntax-preserving methods including code refactoring methods and syntax-breaking including text-oriented methods, is already a good code augmentation method.

Answer to RQ3: Selecting code samples with maximum loss values is the best strategy among our considered variants, i.e., with minimum loss values and random selection. Besides, GenCode with random selection can outperform existing code augmentation methods.

7.4 RQ4: How Does the Data Selection Method Affect the Effectiveness of GenCode?

Table 11 presents the results of accuracy. Overall, LIME demonstrates limited effectiveness and generally underperforms compared to the gradient-based method (i.e., *BADGE*). *Influence Function* achieves better performance in scenarios involving smaller datasets, such as CodeT5 on GCJ and GraphCodeBERT on Refactory. Additionally, *BADGE* exhibits superior performance on LLMs, particularly in tasks such as Qwen2.5-Coder-7B on Refactory and BigCloneBench. Although GenCode outperforms other methods in 12 out of 20 cases,

Table 11 Effectiveness of data augmentation methods w.r.t. test accuracy $\uparrow$ (average $\pm$ standard deviation, %) on original test data

Model	Data Selection	GCJ	Refactory	Java250	BigCloneBench
CodeBERT	LIME	93.67 $\pm$ 0.21	98.81 $\pm$ 0.13	97.25 $\pm$ 0.19	97.45 $\pm$ 0.14
	Influence Function	95.65 $\pm$ 0.19	99.38 $\pm$ 0.09	97.13 $\pm$ 0.24	97.58 $\pm$ 0.19
	BADGE	96.15 $\pm$ 0.13	99.34 $\pm$ 0.22	98.21 $\pm$ 0.17	98.95 $\pm$ 0.12
	GenCode	96.23 $\pm$ 0.18	99.32 $\pm$ 0.24	98.89 $\pm$ 0.09	98.93 $\pm$ 0.15
	LIME	94.98 $\pm$ 0.07	98.12 $\pm$ 0.17	98.01 $\pm$ 0.09	97.83 $\pm$ 0.09
GraphCodeBERT	Influence Function	97.98 $\pm$ 0.11	99.46 $\pm$ 0.02	98.79 $\pm$ 0.08	97.79 $\pm$ 0.13
	BADGE	96.88 $\pm$ 0.07	98.65 $\pm$ 0.06	99.03 $\pm$ 0.11	98.41 $\pm$ 0.04
	GenCode	97.93 $\pm$ 0.14	99.45 $\pm$ 0.17	99.16 $\pm$ 0.13	99.23 $\pm$ 0.09
	LIME	96.36 $\pm$ 0.11	98.45 $\pm$ 0.06	99.03 $\pm$ 0.04	98.21 $\pm$ 0.13
	Influence Function	98.97 $\pm$ 0.09	99.23 $\pm$ 0.13	99.29 $\pm$ 0.13	99.31 $\pm$ 0.12
CodeT5	BADGE	98.86 $\pm$ 0.18	99.54 $\pm$ 0.16	99.36 $\pm$ 0.03	99.33 $\pm$ 0.18
	GenCode	98.85 $\pm$ 0.16	99.64 $\pm$ 0.37	99.38 $\pm$ 0.11	99.34 $\pm$ 0.12
	LIME	96.96 $\pm$ 0.08	99.13 $\pm$ 0.04	99.25 $\pm$ 0.09	99.32 $\pm$ 0.04
	Influence Function	97.35 $\pm$ 0.05	99.41 $\pm$ 0.06	99.31 $\pm$ 0.05	99.34 $\pm$ 0.03
	BADGE	98.96 $\pm$ 0.14	99.65 $\pm$ 0.04	99.39 $\pm$ 0.12	99.35 $\pm$ 0.06
StarCoder2-7B	GenCode	98.93 $\pm$ 0.04	99.65 $\pm$ 0.02	99.41 $\pm$ 0.05	99.36 $\pm$ 0.09
	LIME	97.46 $\pm$ 0.05	99.25 $\pm$ 0.12	99.04 $\pm$ 0.02	99.32 $\pm$ 0.13
	Influence Function	98.13 $\pm$ 0.07	99.52 $\pm$ 0.05	99.12 $\pm$ 0.07	99.35 $\pm$ 0.02
	BADGE	98.98 $\pm$ 0.06	99.49 $\pm$ 0.03	99.38 $\pm$ 0.02	99.37 $\pm$ 0.03
	GenCode	98.96 $\pm$ 0.04	99.65 $\pm$ 0.01	99.42 $\pm$ 0.04	99.36 $\pm$ 0.05
Qwen2.5-Coder-7B	GenCode				

The best results are highlighted in blue. Code understanding tasks include **Authorship attribution** (GCJ), **Bug detection** (Refactory), **Problem classification** (Java250), and **Clone detection** (BigCloneBench)

Table 12 Effectiveness of data augmentation methods w.r.t. test accuracy $\uparrow$ (average $\pm$ standard deviation, %) on transformed test data

Model	Data Selection	GCJ	Refactory	Java250	BigCloneBench
CodeBERT	LIME	89.13 $\pm$ 0.07	95.41 $\pm$ 0.11	93.21 $\pm$ 0.14	91.68 $\pm$ 0.12
	Influence Function	90.34 $\pm$ 0.05	95.51 $\pm$ 0.07	93.35 $\pm$ 0.12	92.17 $\pm$ 0.15
	BADGE	91.31 $\pm$ 0.09	95.61 $\pm$ 0.05	93.61 $\pm$ 0.03	92.64 $\pm$ 0.11
	GenCode	91.23 $\pm$ 0.14	95.63 $\pm$ 0.14	93.62 $\pm$ 0.09	92.56 $\pm$ 0.15
GraphCodeBERT	LIME	88.24 $\pm$ 0.06	95.35 $\pm$ 0.07	94.24 $\pm$ 0.02	93.62 $\pm$ 0.08
	Influence Function	88.97 $\pm$ 0.08	95.62 $\pm$ 0.03	94.32 $\pm$ 0.07	93.58 $\pm$ 0.12
	BADGE	90.01 $\pm$ 0.04	95.88 $\pm$ 0.05	94.87 $\pm$ 0.11	93.96 $\pm$ 0.07
	GenCode	89.78 $\pm$ 0.16	95.67 $\pm$ 0.15	94.89 $\pm$ 0.08	93.89 $\pm$ 0.11
CodeT5	LIME	90.45 $\pm$ 0.13	95.85 $\pm$ 0.13	94.32 $\pm$ 0.04	93.43 $\pm$ 0.05
	Influence Function	91.04 $\pm$ 0.16	95.94 $\pm$ 0.11	94.68 $\pm$ 0.09	93.65 $\pm$ 0.09
	BADGE	91.35 $\pm$ 0.14	96.14 $\pm$ 0.15	95.14 $\pm$ 0.09	93.95 $\pm$ 0.14
	GenCode	91.38 $\pm$ 0.09	95.87 $\pm$ 0.11	95.06 $\pm$ 0.12	94.03 $\pm$ 0.12
StarCoder2-7B	LIME	92.72 $\pm$ 0.16	94.56 $\pm$ 0.14	92.67 $\pm$ 0.12	93.91 $\pm$ 0.09
	Influence Function	92.89 $\pm$ 0.03	95.13 $\pm$ 0.19	92.86 $\pm$ 0.11	94.12 $\pm$ 0.15
	BADGE	92.93 $\pm$ 0.04	96.24 $\pm$ 0.17	93.12 $\pm$ 0.13	94.19 $\pm$ 0.19
	GenCode	92.91 $\pm$ 0.03	96.32 $\pm$ 0.12	93.03 $\pm$ 0.04	94.21 $\pm$ 0.12
Qwen2.5-Coder-7B	LIME	92.94 $\pm$ 0.07	95.13 $\pm$ 0.04	92.68 $\pm$ 0.12	93.87 $\pm$ 0.05
	Influence Function	92.88 $\pm$ 0.19	95.78 $\pm$ 0.09	92.76 $\pm$ 0.08	94.19 $\pm$ 0.06
	BADGE	93.09 $\pm$ 0.05	96.19 $\pm$ 0.02	92.97 $\pm$ 0.11	94.32 $\pm$ 0.05
	GenCode	93.12 $\pm$ 0.02	96.17 $\pm$ 0.15	92.93 $\pm$ 0.09	94.38 $\pm$ 0.14

these results suggest that exploring combinations of existing data selection strategies could be a promising direction for future research.

An interesting observation from the natural robustness results in Table 12 is that, in most cases, the gradient-based method achieves better robustness improvements than GenCode. Specifically, *BADGE* outperforms GenCode by integrating gradient-based uncertainty with diversity via K-means clustering, thereby facilitating the selection of informative and representative samples (Ash et al. 2019).

Answer to RQ4: GenCode achieves better improvements in accuracy compared to other data selection methods. However, the gradient-based method demonstrates superior performance in enhancing robustness. Investigating the trade-offs between GenCode and gradient-based methods presents an interesting direction for future research.

8 Discussion

8.1 Computational Cost of Augmented Data

We conduct a case study to evaluate the computational cost of model training with different data augmentation methods. The results are visualized in Fig. 6. We observe that GenCode incurs the highest computational cost compared to other data augmentation methods including code refactoring, text-oriented, and MixCode. Interestingly, text-oriented data augmentation methods consistently exhibit lower training-time cost than code refactoring methods. In particular, Random Deletion (RD) achieves the best trade-off between performance and efficiency.

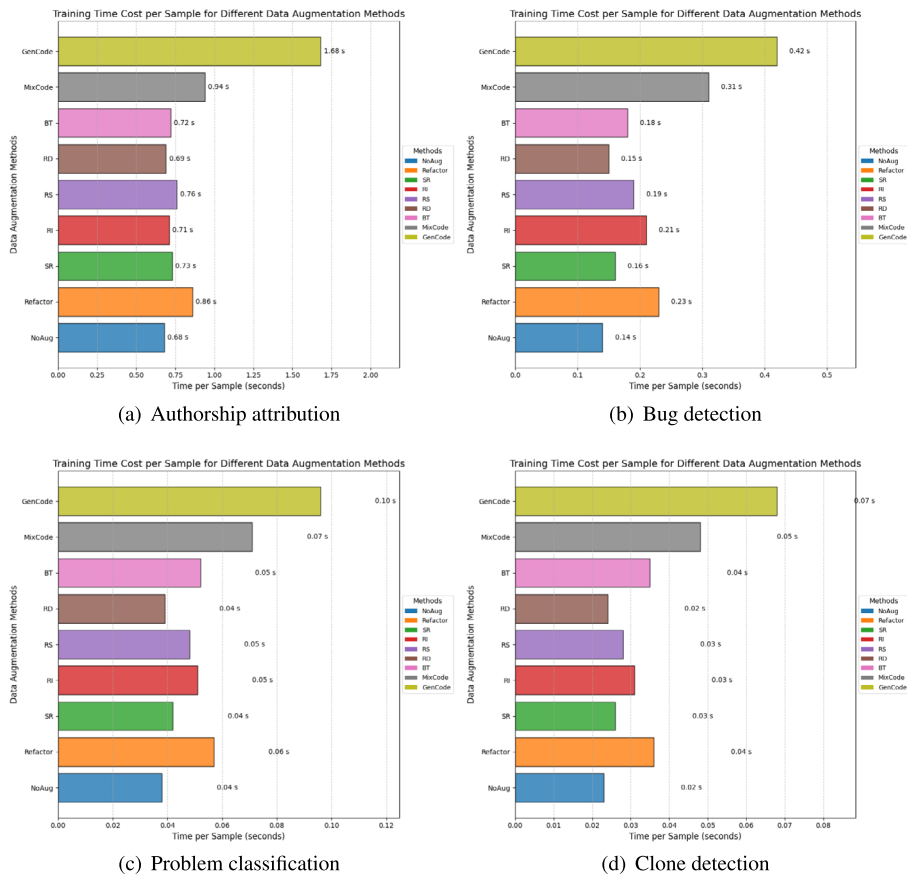


Fig. 6 Computational cost of CodeBERT using different code augmentation methods in each task

Although Back Translation (BT) performs relatively well across some tasks (e.g., authorship attribution and bug detection), it requires significant preprocessing time to generate the augmented data before model training, making it less practical. Therefore, we do not recommend BT as a preferred augmentation strategy in time-sensitive scenarios.

In conclusion, rule-based augmentation methods (e.g., code refactoring or text-oriented data augmentation methods) typically require less computation during model training, but may incur the preprocessing overhead. In contrast, sample-based methods such as MixCode do not require data preparation in advance and can be flexibly applied during training. While GenCode consistently achieves the best accuracy and robustness, it comes with the highest computational cost. Exploring the trade-off between performance and efficiency is an important direction for future work.

8.2 On the Data Distribution of Augmented Data

One common concern in data augmentation is the risk of introducing samples that deviate from the original data distribution, potentially harming model generalization. While

augmented data may not strictly follow the original distribution, prior studies (e.g., Mixup (Zhang et al. 2018), CutMix (Yun et al. 2019)) have shown that moderate, controlled deviations can actually enhance generalization. To mitigate distributional shift, GenCode follows a principled two-stage pipeline: (1) controlled data generation, and (2) loss-based importance-guided selection. In the first stage, GenCode applies a combination of semantic-preserving code refactorings (e.g., variable renaming, field enhancement) and lightweight text-oriented data augmentation methods (e.g., random swap, synonym replacement). These transformations are designed to preserve or minimally alter program semantics, ensuring that the generated data remains close to the original distribution.

In the second stage, GenCode evaluates each augmented sample based on its training loss under the current model. Only samples with the highest loss, those that are potentially more informative, are selected for training, while others are discarded. This loss-based filtering mechanism promotes alignment with the model's effective learning region and helps avoid training on overly noisy or unrepresentative data.

8.3 Limitations

Even though the accuracy of our studied code models already achieved relatively high accuracy in these tasks, i.e., most of them have more than 90% accuracy, their robustness is still low. Regarding the large language models (LLMs), we have checked the literature and found that the clone detection task is still challenging for LLMs, e.g., ChatGPT has only 0.784 Precision for Java-Java pair detection (Khajezade et al. 2024). For other program understanding tasks that have a large number of classes, e.g., the Java250 dataset used in our experiments has 250 classes. It is also hard to use LLMs such as ChatGPT to predict the label since we need to add many label guides in the prompt.

8.4 Threats to Validity

The internal threat to validity mainly comes from the implementation of standard model training and data augmentation methods. The code that is used in our experiment includes 1) the authorship attribution task (Dataset: GCJ) taken from the existing work (Yang et al. 2022), 2) the bug detection task (Dataset: Refactory) adopted from Hu et al. (2019), 3) the program classification task (Dataset: Java250) sourced from Project CodeNeet (Puri et al. 2021), 4) the clone detection task (Dataset: BigCloneBench) chosen from the work (Svajlenko et al. 2014). The implementation of *SR*, *RI*, *RS*, *RD*, and *BT* source from the newest release (Marivate and Sefara 2020), again we adapt them to our experiment. The implementation of *MixCode* and *Refactor* come from its original release (Dong et al. 2023).

The external threats to validity in our study are associated with the code understanding tasks, datasets, pre-trained code models, and data augmentation methods we have selected. We have considered four distinct code understanding tasks: authorship attribution, bug detection, problem classification, and clone detection, utilizing a total of four datasets for these tasks. Our study encompasses two widely used programming languages in the software community, namely Java and Python. Furthermore, we employ three types of pre-trained code models, namely *CodeBERT*, *GraphCodeBERT*, and *CodeT5*, as well as two recently released code-specific large language models, StarCoder2 and Qwen2.5-Coder, in our experiments. For data augmentation methods from NLP, we have selected established

techniques based on their widespread citation in academic papers and their recognition in authoritative journals and conferences.

The construct threats to validity in our study primarily come from various factors, including parameter settings, randomness, and the choice of evaluation measures. For MixCode, the primary parameter is λ , which governs the weight of mixing two feature vectors. We have adhered to the original recommendation for this parameter. Similarly, for data augmentation methods from NLP, we have used the parameters as per their original release. To mitigate the influence of randomness, we have conducted each experiment five times and reported the results as an average along with the standard deviation. In terms of evaluation metrics, our study encompasses both accuracy and robustness. While accuracy assesses the basic performance of pre-trained code models, robustness is employed to evaluate their generalization capabilities.

9 Related Work

Data augmentation has made remarkable success in the field of machine learning, e.g., natural language processing (Feng et al. 2021) and code model (Zhuo et al. 2023). Indeed, data augmentation has proven to be a valuable technique for improving model performance, specifically increasing trained model accuracy and enhancing dataset diversity (Rebuffi et al. 2021). Inspired by that code can be represented as text data (Allamanis et al. 2018), we review related works from three perspectives, deep learning for code understanding, data augmentation for code models, and data augmentation for *NLP*.

9.1 Deep Learning for Code Understanding

Code understanding is important for software developers during the development and maintenance phases. Benefiting from the potential of using deep learning models to understand natural language texts, researchers from the SE borrow the same idea to build deep learning models for code understanding. Code2vec (Alon et al. 2019b) is the very first work that introduced the use of DNN models to learn vector information extracted by code snippets. Code2vec achieved promising results in understanding code on the function name prediction task. Later on, CodeBERT (Feng et al. 2020), was proposed for more code-understanding tasks. CodeBERT follows the pre-training and fine-tuning paradigm. That means it uses multiple programming languages to pre-train the model to learn generation code information. Then, people can fine-tune the pre-trained CodeBERT on a specific downstream task for real usage. CodeBERT achieved impressive results on many code understanding such as comments generation. After that, multiple pre-trained code models have been proposed with ever better performance, such as GraphCodeBERT (Guo et al. 2020), GraphCode2Vec (Ma et al. 2022), and CodeT5 (Wang et al. 2021). More recently, large language models such as ChatGPT⁷ have been demonstrated to be the best choice to solve code understanding tasks.

Different from the above works that focused on designing new model architectures to understand code. We target a more general problem – how to prepare the training data to further boost the training of code models. Our approach is suitable for any code understanding model.

⁷<https://chat.openai.com/>

9.2 Data Augmentation for Code Models

Inspired by the success of data augmentation, researchers have recently dedicated significant efforts to applying data augmentation techniques to large-scale code-related tasks to enhance the performance of code models, focusing on accuracy and robustness. Adversarial training (Goodfellow et al. 2015), which involves generating a set of adversarial examples and incorporating them into the training data, has been explored as a data augmentation method in code learning. To enhance the performance of deep comment generation models, Zhang et al. (2021) proposed a code augmentation method that utilizes the Metropolis-Hastings Modifier (MHM) algorithm to generate the adversarial training dataset. Similar to their work, Mi et al. (2021) generated additional data using Auxiliary Classifier Generative Adversarial Networks (GANs). Besides, as a method that is tailored specifically for code-related tasks, code refactoring which involves restructuring and optimizing code without changing its external behavior is becoming a mainstream code augmentation. Yu et al. (2022) devised program transformation rules tailored for Java and conducted an extensive assessment of their effectiveness when employed as code augmentation techniques across three significant code-related tasks. Allamanis et al. (2021) applied four simple code rewrite rules to improve the efficiency of model training in the program repair task. Furthermore, taking inspiration from *Mixup*, which linearly mixes the features of two distinct image datasets, Dong et al. (2023) proposed *Mixcode* that linearly interpolates the features derived from a pair of programs along with their corresponding labels, wherein One of the programs undergoes code refactoring transformation, while the other remains unchanged. Li et al. (2022) also introduced alternative interpolation methods for the code model, namely binary interpolation and linear extrapolation.

Unlike previous work, our study is the first one that considers the importance of code data for enhancing the performance of code understanding tasks. Specifically, during each epoch, the most valuable data selected by the metric score are fed into the code model for enhancing the performance of training.

9.3 Data Augmentation for NLP

Data augmentation methods that are used in NLP can be roughly divided into three categories, namely *paraphrasing*, *noising-based methods*, and *sampling-based methods* (Feng et al. 2021). Xie et al. (2020) proposed the back-translation technique that involves bidirectionally applying a translation model to each sentence. More precisely, this method first translates the original text into another language and then reverts it back to the original one. Paraphrasing is widely employed in the field of NLP since it can convey identical information as the original text but in a different form. Wei and Zou (2019) applied a series of noising-based methods named *EDA* to augment the text data. Noising-based methods generally involve the subtle addition of noise to the original data while preserving their semantic information. For example, *Random Swap* (one of the methods from *EDA*) can include randomly selecting two words in a sentence and swapping their positions. Besides, different from paraphrasing or noising-based methods, sampling-based methods are task-specific and necessitate both data and their label formats. Also inspired by the *Mixup*, Guo et al. (2019) adapted it from its original use with image data to text data. They conducted an empirical study to investigate its effectiveness. Specifically, they generated new synthetic data by lin-

early mixing the embeddings instead of directly operating on the raw text data. Furthermore, to overcome the limitations of traditional methods such as *Synonym Replacement*, which generally require significant prior knowledge and are prone to sub-optimal results, Ren et al. (2021) proposed *Text AutoAugment*. In detail, it considers a combination of various text-oriented data augmentation operations as an augmentation policy and employs an efficient Bayesian Optimization algorithm to automatically search for the best policy.

In contrast to the data augmentation methods used in NLP, our work focuses on the strategy that guides code model training. The strategy involves both existing data augmentation methods derived directly from code data and those adapted from text data. Moreover, our approach also allows for the integration of new program transformation methods that researchers may develop in the future.

10 Conclusion

We propose the first generation-and-selection-based framework, GenCode, to boost code understanding models. The basic assumption behind GenCode is that less confident data contributes more to the model training. Based on this assumption, GenCode utilizes atomic data augmentation operators to generate candidate training data first and selects data with maximum loss values to train the code model. Our evaluation demonstrates that GenCode can produce more accurate and robust models than MixCode. Besides, we analyze how the influence score affects the effectiveness of GenCode.

Author Contributions **Zeming Dong**: Conceptualization, Methodology, Software, Data curation, Investigation, Formal analysis, Experiments, Visualization, Writing – original draft, Writing – review & editing. **Qiang Hu**: Methodology, Validation, Writing – review & editing. **Xiaofei Xie**: Supervision, Validation, Methodology. **Maxime Cordy**: Supervision, Methodology, Funding acquisition. **Mike Papadakis**: Supervision, Methodology, Funding acquisition. **Yves Le Traon**: Supervision, Methodology, Funding acquisition. **Jianjun Zhao**: Supervision, Validation, Methodology.

Funding This work is supported by the Luxembourg National Research Fund (FNR) through the CORE project under Grant C22/IS/17426831/MeMoRIA.

Data Availability The datasets generated during and/or analyzed during the current study are available in the: <https://github.com/zemingd/GenCode>.

Declarations

Funding This work is supported by the Luxembourg National Research Fund (FNR) through the CORE project under Grant C22/IS/17426831/MeMoRIA.

Ethical Approval Not applicable.

Informed Consent Not applicable.

Conflict of Interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Clinical Trial Number Not applicable.

References

- Allamanis M, Barr ET, Devanbu P, Sutton C (2018) A survey of machine learning for big code and naturalness. *ACM Comput Surv (CSUR)* 51(4):1–37
- Allamanis M, Jackson-Flux HR, Brockschmidt M (2021) Self-supervised bug detection and repair. In: Beygelzimer A, Dauphin Y, Liang P, Vaughan JW (eds) *Advances in neural information processing systems*
- Alon U, Brody S, Levy O, Yahav E (2019a) code2seq: generating sequences from structured representations of code. In: *International conference on learning representations*
- Alon U, Zilberstein M, Levy O, Yahav E (2019b) code2vec: learning distributed representations of code. *Proc ACM Progr Lang* 3(POPL):1–29
- Ash JT, Zhang C, Krishnamurthy A, Langford J, Agarwal A (2019) Deep batch active learning by diverse uncertain gradient lower bounds. *arXiv preprint arXiv:1906.03671*
- Bielik P. and Vechev M (2020) Adversarial robustness for code. In: III HD, Singh A (eds) *In: Proceedings of the 37th international conference on machine learning volume 119 of proceedings of machine learning research, PMLR*, pp 896–907
- Bui ND, Yu Y, Jiang L (2021) Self-supervised contrastive learning for code retrieval and summarization via semantic-preserving transformations. In: *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval SIGIR '21, New York NY USA. Association for Computing Machinery*, pp 511–521
- Cubuk ED, Zoph B, Shlens J, Le QV (2020) Randaugment: Practical automated data augmentation with a reduced search space. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, pp 702–703
- de Paula Rodrigues GE, Braga AM, Dahab R (2023) Detecting cryptography misuses with machine learning: Graph embeddings transfer learning and data augmentation in source code related tasks. *IEEE Trans Reliab*
- Dinella E, Dai H, Li Z, Naik M, Song L, Wang K (2020) Hoppity: Learning graph transformations to detect and fix bugs in programs. In: *international conference on learning representations*
- Ding B, Qin C, Zhao R, Luo T, Li X, Chen G, Xia W, Hu J, Tuan LA, Joty S (2024) Data augmentation using llms: Data perspectives learning paradigms and challenges. In *findings of the association for computational linguistics ACL 2024*:1679–1705
- Dong Z, Hu Q, Zhang Z, Zhao J (2024) On the effectiveness of graph data augmentation for source code learning. *Knowl-Based Syst* 285:111328
- Dong Z, Hu Q, Guo Y, Zhang Z, Cordy M, Papadakis M, Le Traon Y, Zhao J (2025) Boosting source code learning with text-oriented data augmentation: An empirical study. *Empir Softw Eng* 30(3):68
- Dong Z, Hu Q, Zhang Z, Guo Y, Cordy M, Papadakis M, Le Traon Y, Zhao J (2024a) On the effectiveness of hybrid pooling in mixup-based graph learning for language processing. *J Syst Softw* 112139
- Dong Z, Hu Q, Zhang Z, Zhao J (2024b) On the effectiveness of graph data augmentation for source code learning. *Knowl-Based Syst* 285:111328
- Feng SY, Gangal V, Wei J, Chandar S, Vosoughi S, Mitamura T, Hovy E (2021) A survey of data augmentation approaches for NLP. In: *Findings of the association for computational linguistics: ACL-IJCNLP 2021, Online. Association for Computational Linguistics*, pp 968–988
- Feng Z, Guo D, Tang D, Duan N, Feng X, Gong M, Shou L, Qin B, Liu T, Jiang D, Zhou M (2020) Codebert: A pre-trained model for programming and natural languages, pp 1536–1547
- Gao X, Saha RK, Prasad MR, Roychoudhury A (2020) Fuzz testing based data augmentation to improve robustness of deep neural networks. In: *Proceedings of the ACM/IEEE 42nd international conference on software engineering ICSE '20, New York NY USA. Association for Computing Machinery*, pp 1147–1158
- Goodfellow IJ, Shlens J, Szegedy C (2015) Explaining and harnessing adversarial examples. In: Bengio Y, LeCun Y (eds) *3rd international conference on learning representations (ICLR)*
- Guo D, Ren S, Lu S, Feng Z, Tang D, Liu S, Zhou L, Duan N, Svyatkovskiy A, Fu S et al (2020) Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366*
- Guo H, Mao Y, Zhang R (2019) Augmenting data with mixup for sentence classification: An empirical study
- Hu Q, Guo Y, Xie X, Cordy M, Ma L, Papadakis M, Le Traon Y (2024) Active code learning: Benchmarking sample-efficient training of code models. *IEEE Trans Softw Eng*
- Hu Y, Ahmed UZ, Mechtav S, Leong B, Roychoudhury A (2019) Re-factoring based program repair applied to programming assignments. In *34th IEEE/ACM international conference on automated software engineering (ASE)*, IEEE, pp 388–398
- Hui B, Yang J, Cui Z, Yang J, Liu D, Zhang L, Liu T, Zhang J, Yu B, Lu K et al (2024) Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*

- Khajezade M, Wu JJ, Fard FH, Rodríguez-Pérez G, Shehata MS (2024) Investigating the efficacy of large language models for code clone detection. In: Proceedings of the 32nd IEEE/ACM international conference on program comprehension, pp 161–165
- Kingma DP, Ba J (2014) Adam: a method for stochastic optimization. arXiv preprint [arXiv:1412.6980](https://arxiv.org/abs/1412.6980)
- Koh PW, Liang P (2017) Understanding black-box predictions via influence functions. In: International conference on machine learning, PMLR, pp 1885–1894
- Lacerda G, Petrillo F, Pimenta M, Guéhéneuc YG (2020) Code smells and refactoring: A tertiary systematic review of challenges and observations. *J Syst Softw* 167:110610
- Li H, Miao C, Leung C, Huang Y, Huang Y, Zhang H, Wang Y (2022) Exploring representation-level augmentation for code search. In EMNLP, pp 4924–4936
- Lozhkov A, Li R, Allal LB, Cassano F, Lamy-Poirier J, Tazi N, Tang A, Pykhtar D, Liu J, Wei Y et al (2024) Starcoder 2 and the stack v2: The next generation. arXiv preprint [arXiv:2402.19173](https://arxiv.org/abs/2402.19173)
- Lu S, Guo D, Ren S, Huang J, Svyatkovskiy A, Blanco A, Clement C, Drain D, Jiang D, Tang D et al (2021) Codexglue: A machine learning benchmark dataset for code understanding and generation. In: 35th conference on neural information processing systems (NeurIPS)
- Maćkiewicz A, Ratajczak W (1993) Principal components analysis (PCA). *Comput Geosci* 19(3):303–342
- Maćkiewicz A, Ratajczak W (1993) Principal components analysis (PCA). *Comput Geosci* 19(3):303–342
- Marivate V, Sefara T (2020) Improving short text classification through global augmentation methods. In: machine learning and knowledge extraction: 4th IFIP TC 5 TC 12 WG 8.4 WG 8.9 WG 12.9 International cross-domain conference CD-MAKE 2020 Dublin Ireland August 25–28 2020 Proceedings 4, Springer, pp 385–399
- Mi Q, Xiao Y, Cai Z, Jia X (2021) The effectiveness of data augmentation in code readability classification. *Inf Softw Technol* 129:106378
- Park S, Kim Y, Han YS (2023) Contrastive learning with keyword-based data augmentation for code search and code question answering. In: Vlachos A, Augenstein I (eds) Proceedings of the 17th conference of the european chapter of the association for computational linguistics, Dubrovnik Croatia. Association for Computational Linguistics, pp 3609–3619
- Pour MV, Li Z, Ma L, Hemmati H (2021) A search-based testing framework for deep neural networks of source code embedding. In 14th IEEE conference on software testing verification and validation (ICST), Los Alamitos CA USA. IEEE Computer Society, pp 36–46
- Puri R, Kung DS, Janssen G, Zhang W, Domeniconi G, Zolotov V, Dolby J, Chen J, Choudhury M, Decker L et al (2021) Codenet: A large-scale ai for code dataset for learning a diversity of coding tasks
- Rebuffi SA, Gowal S, Calian DA, Stimberg F, Wiles O, Mann TA (2021) Data augmentation can improve robustness. *Adv Neural Inf Process Syst* 34:29935–29948
- Ren S, Zhang J, Li L, Sun X, Zhou J (2021) Text AutoAugment: Learning compositional augmentation policy for text classification. In: Proceedings of the 2021 conference on empirical methods in natural language processing
- Ribeiro MT, Singh S, Guestrin C (2016) "why should i trust you?" explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp 1135–1144
- Schober P, Boer C, Schwarte LA (2018) Correlation coefficients: appropriate use and interpretation. *Anesth Analg* 126(5):1763–1768
- Sun Z, Li L, Liu Y, Du X (2022) On the importance of building high-quality training datasets for neural code search. In: Proceedings of the 44th international conference on software engineering ICSE '22, New York NY USA. Association for Computing Machinery, pp 1609–1620
- Svajlenko J, Islam JF, Keivanloo I, Roy CK, Mia MM (2014) Towards a big data curated benchmark of inter-project code clones. In: 2014 IEEE international conference on software maintenance and evolution, IEEE, pp 476–480
- Verma V, Lamb A, Beckham C, Najafi A, Mitliagkas I, Lopez-Paz D, Bengio Y (2019) Manifold mixup: better representations by interpolating hidden states. In International conference on machine learning, PMLR, pp 6438–6447
- Wang D, Jia Z, Li S, Yu Y, Xiong Y, Dong W, Liao X (2022) Bridging pre-trained models and downstream tasks for source code understanding. In: Proceedings of the 44th international conference on software engineering, New York NY USA. Association for Computing Machinery, pp 287–298
- Wang W, Li G, Ma B, Xia X, Jin Z (2020) Detecting code clones with graph neural network and flow-augmented abstract syntax tree. In: 2020 IEEE 27th international conference on software analysis, evolution and reengineering (SANER), IEEE, pp 261–271
- Wang Y, Wang W, Joty S, Hoi SC (2021) CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In: Proceedings of the 2021 conference on empirical methods in natural language processing, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics, pp 8696–8708

- Wei J, Zou K (2019) Eda: easy data augmentation techniques for boosting performance on text classification tasks. In: Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP), Hong Kong, China. Association for Computational Linguistics, pp 6382–6388
- Wu X, Jang U, Chen J, Chen L, Jha S (2018) Reinforcing adversarial robustness using model confidence induced by adversarial training. In: Dy J, Krause A (eds) In: Proceedings of the 35th international conference on machine learning volume 80 of Proceedings of machine learning research, PMLR, pp 5334–5342
- Xie Q, Dai Z, Hovy E, Luong MT, Le QV (2020) Unsupervised data augmentation for consistency training. In: Proceedings of the 34th international conference on neural information processing systems NIPS'20 Red Hook NY USA. Curran Associates Inc
- Yang G, Zhou Y, Chen X, Zhang X, Han T, Chen T (2023) Exploitgen: Template-augmented exploit code generation based on codebert. *J Syst Softw* 197:111577
- Yang Z, Sinnott RO, Bailey J, Ke Q (2023) A survey of automated data augmentation algorithms for deep learning-based image classification tasks. *Knowl Inf Syst* 65(7):2805–2861
- Yang Z, Sinnott RO, Bailey J, Ke Q (2023b) A survey of automated data augmentation algorithms for deep learning-based image classification tasks. *Knowl Inf Syst* 65(7):2805–2861
- Yasunaga M, Liang P (2020) Graph-based self-supervised program repair from diagnostic feedback. In: Proceedings of the 37th international conference on machine learning ICML'20. JMLR.org
- Yefet N, Alon U, Yahav E (2020) Adversarial examples for models of code. *Proc ACM Progr Lang* 4(OOPSLA):1–30
- Yu S, Wang T, Wang J (2022) Data augmentation by program transformation. *J Syst Softw* 190:111304
- Yun S, Han D, Oh SJ, Chun S, Choe J, Yoo Y (2019) Cutmix: regularization strategy to train strong classifiers with localizable features. In: Proceedings of the IEEE/CVF international conference on computer vision, pp 6023–6032
- Zhang H, Li Z, Li G, Ma L, Liu Y, Jin Z (2020) Generating adversarial examples for holding robustness of source code processing models. In Proceedings of the AAAI conference on artificial intelligence 34:1169–1176
- Zhang H, Li Z, Li G, Ma L, Liu Y, Jin Z (2020) Generating adversarial examples for holding robustness of source code processing models. In: Proceedings of the AAAI conference on artificial intelligence, vol 34, pp 1169–1176
- Zhang X, Zhou Y, Han T, Chen T (2021) Training deep code comment generation models via data augmentation. In: 12th Asia-Pacific symposium on internetware internetware '20, New York NY USA. Association for Computing Machinery, pp 185–188
- Zhou Y, Liu S, Siow J, Du X, Liu Y (2019) Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In: Proceedings of the 33rd international conference on neural information processing systems Red Hook NY USA. Association for Computing Machinery
- Zhuo TY, Yang Z, Sun Z, Wang Y, Li L, Du X, Xing Z, Lo D (2023) Data augmentation approaches for source code models: A survey. *arXiv preprint [arXiv:2305.19915](https://arxiv.org/abs/2305.19915)*

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.